

A Compression Technique to Materialize Transitive Closure

H. V. JAGADISH
AT&T Bell Laboratories

An important feature of database support for expert systems is the ability of the database to answer queries regarding the existence of a path from one node to another in the directed graph underlying some database relation. Given just the database relation, answering such a query is time-consuming, but given the transitive closure of the database relation a table look-up suffices. We present an indexing scheme that permits the storage of the pre-computed transitive closure of a database relation in a compressed form. The existence of a specified tuple in the closure can be determined from this compressed store by a single look-up followed by an index comparison. We show how to add nodes and arcs to the compressed closure incrementally. We also suggest how this compression technique can be used to reduce the effort required to compute the transitive closure.

Categories and Subject Descriptors: F.2.2 [Analysis of Algorithms and Problem Complexity]: Nonnumerical Algorithms and Problems; H.2.4 [Database Management]: Algorithms; I.2.8 [Artificial Intelligence]: Problem Solving, Control Methods and Search

General Terms: Algorithms, Performance, Systems

Additional Key Words and Phrases: Deductive databases, graph theory, hierarchy, inheritance, query processing, reachability, recursion, transitive closure

1. INTRODUCTION

There is a growing interest in extending relational databases to handle recursive queries [1, 7]. It has been established that every linear recursive query can be expressed as a transitive closure (possibly preceded and followed by relational algebraic operations) [22]. Transitive Closure has also appeared as a primitive operation in Zloof's QBE [37], Guttman's Quel* [16], Probe's traversal recursion [31], and α -extended relational algebra [1]. In addition, expert systems are using inheritance more and more (see, for example, [12, 33]). Inheritance is also a central concept in object-oriented and semantic data models (see, for example [18, 23]). These systems require, as basic operations that are performed repeatedly, determining whether there exists a path between a specified pair of points (for example to find if one class is subsumed by another), or determining all

Preliminary ideas from this paper were presented at the Expert Database Systems Conference, Tyson's Corner, Va., April 1988. Author's address: H. V. Jagadish, AT&T Bell Laboratories, Murray Hill, N.J. 07974.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1990 ACM 0362-5915/90/1200-0558 \$01.50

nodes that are reachable from a particular point (for example, to find all inherited properties). In short, transitive closure is likely to be an important operation in the next generation of database systems and machines, and is almost a requirement for any expert database. An important use of transitive closure is in determining whether there exists a path between two specified nodes. In this paper we focus on this aspect of transitive closure.

A major impediment to the provision of recursion in today's database systems is that transitive closure, though a simple, well-understood, fixed-point operation, is extremely expensive to compute. Even if selections are specified in a transitive closure query, answering it still remains expensive since selections applied to the result of the transitive closure can often not be "moved in". (See, for example, [9, 13, 26]). There is considerable effort being devoted currently to devise new and more efficient ways to compute closure [2, 3, 6, 19, 20, 28, 34, 35].

A possible solution to this problem could come from the concept of *materialized views*, which is well-established in the relational database literature as a means for speeding up query processing. Frequently required "views" of the database are kept "materialized" (that is, explicitly computed) so that queries can be satisfied directly from such precomputed views. (See, for example, [11, 17, 27]).

Instead of repeatedly computing frequently required transitive closures, it may be better to precompute the appropriate transitive closures and to store these as "materialized views". Unfortunately, even for fairly small relations, the transitive closure can consist of a phenomenal number of tuples, causing storage requirements to shoot up to unacceptably high levels (see Section 5). In this paper we propose a compression technique that reduces the storage requirement to a small fraction of that required by the complete transitive closure, and yet permits queries related to the closure to be answered efficiently.

For the purposes of this paper it is convenient to think of the database relation as defining a directed graph. Each instance of the field(s) participating in the transitive closure represents a node in this graph, and each tuple in the relation represents an arc. The graph corresponding to the transitive closure relation is the transitive closure of the graph so constructed. For simplicity, we shall assume that the graph is stored at each node followed by a list of its successors. In an actual implementation, the database relation may be stored in some other fashion that is most efficient. However, an index structure must then be built that permits us to refer to all the "successors" of a node in such a graph. For example, a database in a factory may include a "parts" relation, specifying the components required to manufacture a widget, the components needed to manufacture these components, and so forth. For such a relation, each type of component is represented by a node in the corresponding graph. Each tuple, stating that some component is part of another component, corresponds to a directed arc in the graph. The successor list of a component X comprises all the components and assemblies that require X in their manufacture.

The chain decomposition of a transitive closure graph, using nonintersecting chains, has previously been studied in the graph theory literature [30]. In [5], efficient encoding structures are presented for storing the path information justifying each tuple created in the transitive closure graph. In [4], tree decomposition is suggested as a variant on chain decomposition. In [8], there is a

discussion of how to order data to minimize disk accesses during the computation of transitive closure. However, there is no precomputation.

In Section 2, we present the indexing scheme and show how it can be used to store the transitive closure and to determine reachability on a directed graph. In Section 3 we suggest ways to obtain a set of indices for a given graph consistent with the scheme suggested in Section 2, and demonstrate bounds on the sizes of the lists that have to be stored. An extension to the basic scheme is presented in Section 4 that permits further compression at the expense of additional work to answer some queries. In Section 5, simulation results are presented to verify the benefits obtained by our compression strategy.

Whenever a database update is made the effect of the update has to be “propagated” to all the appropriate materialized views of the database that are being maintained. Therefore, the maintenance of materialized views is likely to be most advantageous when updates are relatively infrequent compared to queries. Unless efficient update techniques are found, it may not be worthwhile to maintain a materialized view if updates to the database are expected, even if these be few. In Section 6, we suggest techniques to incrementally update an existing compressed transitive closure.

A large fraction of the cost in computing the transitive closure of a relation is in the I/O between disk and main memory. One should expect that if the transitive closure is compressed then the amount of I/O is also reduced. Even for a main memory computation, having shorter successor lists at each node is beneficial. In Section 7, we show how to compute the closure while compressing it at the same time and present simulation results demonstrating the advantages of so doing. Finally, in Section 8, we summarize our findings and suggest directions for further research.

2. THE INDEXING SCHEME

To each node in a graph, assign a pair of indices, such that the following rule is satisfied.

Indexing Rule. For any pair of nodes a and b with indices (i, j) and (k, j) , if $i < k$, there exists a directed path from a to b .

In that case, nodes a and b are said to be on the same *chain*, which we shall call chain j . Node a is said to *dominate* node b .

The indexing rule states that two nodes may have the same value for the second index in their respective pairs of indices only if there is a path from one of them to the other. Techniques to obtain such an assignment of indices will be considered in Section 3. For the rest of this paper, we shall refer to a node either by its name or by the pair of indices assigned to it, interchangeably.

Let us suppose that the transitive closure of a graph is computed and is available as a list for each node of all nodes reachable from it. We call this list of all nodes reachable from a node a , the *successor list* of node a . (The successor list of a node should be distinguished from its lists of *immediate* successors. The former includes all reachable nodes, the latter only nodes reachable in one step).

Then, from each successor list, delete all nodes that are dominated by other nodes in the successor list. The resulting compressed successor list will have at

most one entry for each chain. We can formally state:

Procedure Compress

```

For each node  $m$  in the graph
  For each entry  $(i, j)$  in the successor list of  $m$ 
    If there exists an entry  $(k, j)$  in the successor list of  $m$  with  $k < i$ 
      Delete  $(i, j)$ 

```

While procedure compress has been stated in the form easiest to understand logically, a literal implementation would not be efficient. A more efficient implementation may simply retain the minimum value for the first index for each value of the second index in the successor list of a node. Only the compressed successor list, obtained by applying procedure compress, need be stored for each node on account of the following theorem.

REACHABILITY THEOREM. *Let a node b have an index pair (i, j) . There exists a directed path from some node a to node b if and only if there exists (k, j) in the compressed successor list of a with $k \leq i$.*

PROOF. If there is such a (k, j) in the successor list of a , then there is a path from a to (k, j) . If $k = i$, then there exists a path from a to b . Suppose that $k < i$. Then, by the Indexing rule, there exists a path from (k, j) to (i, j) , i.e., from (k, j) to node b . Therefore there is a path from node a to node b .

Suppose that there is a path from node a to node b , then (i, j) is in the successor list of node a before compression. (i, j) could be deleted from this list in the course of the compression if and only if there is some other node (k, j) in the successor list of a with $k < i$. Therefore the compressed successor list of a either has (i, j) in it, or has (k, j) in it for some $k < i$. $\square$

Thus, to determine whether a node b is reachable from a node a , first determine the pair of indices associated with b . Then check the successor list of a for an entry with the same value of the second index (a node lying on the same chain) as b . If this entry dominates b , or is b itself, b is reachable from a . If the entry does not dominate b , or does not exist, then b is not reachable from a .

Example 1. Chain Compression. The graph in Figure 1a consists of 7 nodes and 11 arcs. Two chains, A-B-D-F and G-C-E, are required to cover this graph. The transitive closure can be computed and the successor lists at each node compressed to obtain the successor lists shown in Figure 1b. Now suppose that we wish to find out whether B can reach E. We first determine that the indices assigned to E are (2, 1). Then, in the successor list of B, we find (1, 1) as the entry with second index 1. Since the first index of this entry, 1, is less than the first index of the desired destination, 2, we conclude that there exists a path from B to E.

2.1 Finding All Successors

Sometimes one may wish to find all nodes that are reachable from a given node, rather than merely finding out whether a specific point can be reached. For example, one might want to know the names of all ancestors of John rather than just find out whether Mary is John's ancestor. If the data-structures are so

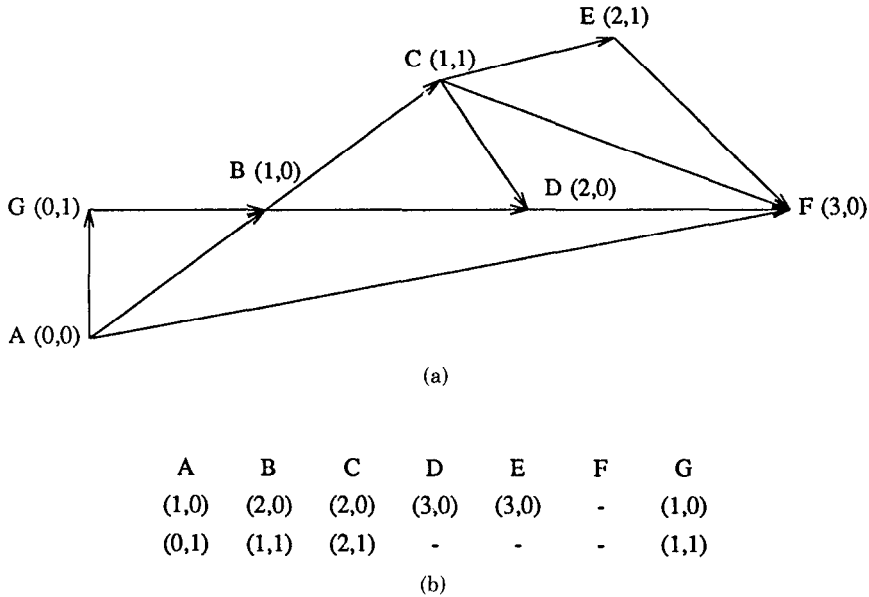


Fig. 1. (a) An acyclic graph with its index assignment shown, and (b) the compressed successor lists for the nodes in this graph.

organized that it is possible to read off all the nodes that lie on a specified chain such queries can be answered with ease. To find all successors of some node a , retrieve the compressed list of successors of a . Each entry in this list belongs to a different chain. For each entry, read off the names of all the succeeding nodes on that chain. The nodes read off are exactly the successors of a .

One may sometimes wish to find nodes that are reachable from a set of nodes rather than a single specified node. A combined compressed successor list can be formed for the specified set S of nodes by applying the following simple procedure:

Procedure Aggregate

- For each value of second index, j (i.e., for each chain)
 - Over the compressed successor lists of all nodes in set S
 - Set i to be the minimum value of the first index.
 - Include (i, j) in the aggregate compressed successor list of S

From this aggregate compressed successor list for the specified set of nodes, one can determine whether any specific destination is reachable from at least one node in the set or one can read off the set of all destinations reachable, just as with an ordinary compressed successor list.

2.2 Look-Up Effort

The compressed transitive closure of a graph, obtained as discussed above, is likely to be significantly smaller than the full transitive closure of the graph. The savings obtained in storage will be explored in depth in subsequent sections of this paper. Here we look briefly at the side benefit obtained in terms of effort.

The effort required to determine whether a node d is reachable from a node s is two look-ups and a comparison in the case of the compressed closure, as opposed to a single look-up in the case of the full closure. However, the single look-up in the latter case is from a much larger set, so that, depending on the access methods used, the effort may actually be less in the case of a compressed closure. We analyze these costs below for some assumed data structures. The actual costs will vary depending on the exact implementation used.

The simplest possible data structure is a two-dimensional array, with an entry for each source-destination node pair. This entry is True if there exists a path from the specified source node to the specified destination node, and is false otherwise. Whether a specified source can reach a specified destination is a constant time look-up in the array. The corresponding data structure for a compressed closure is a two-dimensional array with an entry for each source-chain pair. Each entry has the identification of the most dominant node in the chain reachable from the specified source. The index assignments to each node are stored in a separate array. The effort is two constant-time look-ups, one in each array, and a comparison. In a graph with n nodes, covered with c chains, the full closure requires n^2 words of storage, while the compressed closure requires $n(c + 2)$ words (nc for the two-dimensional array, and $2n$ for the array with index assignments).

Due to the large storage requirements, a simple array is not practical. Consider the transitive closure stored as a relation in a database, with a source field and a destination field. Let this relation be indexed on the source field as the primary key. Assume that there is no secondary index but that the relation is clustered on the source field. In a graph with n nodes, the effort to traverse the index is $O(\log n)$, to be followed by the $O(n)$ effort of examining all tuples with the specified source field.

With chain compression, the effort to locate the successor list is the same as in the case of no compression, $O(\log n)$, but since the successor lists themselves are smaller, fewer pages will have to be read from disk, especially in the absence of clustering. The linear scan within a successor list can be faster since the list is smaller. The total effort is $O(\log n + c)$. If c , the number of chains, is sufficiently small, each successor list can be stored as an array instead of a list, making the search for the destination a constant-time look-up rather than a scan.

Indexing the full transitive closure on both source and destination fields is impractical since the index structure would have to contain the information about whether any particular source can reach a particular destination. Such an index structure would itself be $O(n^2)$ in size.

3. ASSIGNMENT OF INDICES

The previous section assumed that a set of index pairs satisfying the *Indexing Rule* had been assigned to all the nodes in the graph. All arguments there would apply irrespective of how such an index assignment was obtained. For instance, if each node were assigned a unique second index, then the *Indexing Rule* would be satisfied trivially. However, no node would dominate any other and no compression would be achieved. The purpose of this section is to show how to obtain a "good" index assignment.

The choice of the word “chain” for the second index is meant to evoke an intuition of a set of adjacent edges forming a path in the graph from an arbitrary start node to an arbitrary finish node. Let each such “physical chain” mark the nodes that it passes through. A chain cover for the graph is found when each node has been marked by at least one such chain. If more than one such “physical chain” passes through a node, we have two options. We could assign multiple sets of indices to the node in question. A second option is to ensure a unique pair of indices for each node by “breaking” all but one of the chains that pass through the node, so that they do not include the node. To simplify the storage of indices, we prefer the second option. Thus, a chain, as defined by the second index assigned to the nodes, may consist of disjoint pieces of a physical chain with some intermediate nodes deleted.

Example 2. Chain Definition. Consider the graph of Figure 1a again. Two physical chains were chosen to cover the graph initially. These were A-B-D-F and G-B-C-E. Notice that the node B occurs on both physical chains. Indices are assigned to it only by virtue of B occurring on chain 0. Node B is deleted from the other chain (chain 1) resulting in a chain G-C-E as if there were a direct link from G to C.

3.1 A Tight Bound for Acyclic Graphs

Given a graph $G(V, E)$, the *width* of G is the cardinality of the largest set $U \subseteq V$, such that for every pair of nodes $x, y \in U$, there does not exist a path from x to y or from y to x . In other words, the *width* of a graph is the maximum number of nodes that can be selected from the graph such that for every pair of nodes in the selected set it is not possible to reach one from the other.

WIDTH THEOREM. *For a directed acyclic graph:*

- (i) *There exists an index assignment satisfying the Indexing Rule such that the number of chains in the graph is equal to its width, and*
- (ii) *There cannot be an index assignment satisfying the Indexing Rule with a smaller number of chains.*

PROOF (after Dilworth [14]). Given a directed acyclic graph G , define $\alpha(G)$ to be the set of all nodes in G that have no predecessors. Define $\omega(G)$ to be the set of all nodes in G that have no successors. Let the width of G be f . Let $\phi(G)$ be a set of f nodes from G that are all mutually unreachable. (In other words, $\phi(G)$ is a set, not necessarily unique, that determines the width of the graph).

The first part of the theorem is constructively established by finding a set of f chains that cover all the vertices of G . The construction is by reducing the graph recursively in one of two ways. Either a chain is established and all vertices covered by this chain removed from further consideration, or the graph is divided in two, and chain covers found independently for the two parts. Since both actions reduce the size of the graph (measured, say, in number of nodes) for which the problem must be solved, the recursion will terminate for every finite graph. Thus the proof is by backward induction as the number of chains is reduced, with the added twist that the graph may sometimes be split and the induction continued independently on each of the resulting parts.

Suppose that the only choices for $\phi(G)$ and $\alpha(G)$ and/or $\omega(G)$. Choose some node $x \in \alpha(G)$ and some node $y \in \omega(G)$, such that y can be reached from x or $x = y$. (It is always possible to find such a pair in a directed acyclic graph). Establish a chain from x to y . (The chain has one node and no edges if $x = y$). In general, the chain is $x-z_1-z_2-\dots-y$. For each node z_i on this chain, create an edge from each immediate predecessor of z_i to each immediate successor of z_i . Delete all nodes on the chain, along with any associated edges. The new edges created are in the transitive closure of G , and do not contribute to a path between any source-destination pair between which there was not a path in G . The deleted chain covers at least one node, hence at least one node is deleted. Since the only sets that could determine the width of graph have been reduced by one element, the width of G must have been reduced from f to $f - 1$. Call the resulting graph G' . Solve the problem of finding $f - 1$ chains to cover the graph G' , recursively.

Suppose it is not true that the only width determining sets are $\alpha(G)$ and/or $\omega(G)$. Find a $\phi(G)$ different from $\alpha(G)$ and $\omega(G)$. Divide G into two subgraphs, based upon $\phi(G)$. One subgraph comprises all nodes that can reach one or more nodes in $\phi(G)$. The other subgraph has all nodes that can be reached by one or more nodes in $\phi(G)$. (Every node in G is not in $\phi(G)$ has a directed path either to or from $\phi(G)$. Any node that does not have such a path can be added to the width determining set thereby increasing the width of the graph and violating our assumption that $\phi(G)$ is a width determining set). Additionally, $\phi(G)$ itself is included in both subgraphs. Every edge between two nodes within the same sub-graph is retained. Every edge between a node in one subgraph and a node in a different subgraph is deleted. If the width of G is f , the width of each subgraph created is also f . Call the subgraphs G_1 and G_2 respectively. Solve the problem of finding f chains to cover a graph, separately for G_1 and G_2 . Once the problem is solved, through recursive application of this and the previous step, for each subgraph, the desired chain cover for G is obtained by concatenating the chains used to cover G_1 and G_2 . The width determining set $\phi(G) = \omega(G_1) = \alpha(G_2)$. Therefore, exactly one chain in G_1 terminates at each node in $\phi(G)$. Similarly, exactly one chain in G_2 emanates from each node in $\phi(G)$. For each node in $\phi(G)$, these two chains can be concatenated.

By construction, we have shown that it is possible to obtain a set of no more than f chains such that each node in G lies on exactly one chain. Thus the first part of the theorem is established. The second part of the theorem is easy. If two nodes are mutually unreachable, then they cannot lie on the same chain. Since G is of width f , it has f mutually unreachable nodes, each of which must lie on a different chain. Therefore, there must be at least f chains in a correct index assignment. $\square$

3.2 Obtaining an Optimal Index Assignment for Acyclic Graphs

The storage required for the compressed transitive closure of a graph with n nodes covered by c chains is $O(nc)$. The proof of the Width Theorem suggests a technique for finding an index assignment that is optimal in the sense that it requires the smallest number of chains. Unfortunately, the constructive procedure given is cumbersome since it involves repeatedly determining the width (and all possible choices of $\phi(\cdot)$) for the given graph G and various subgraphs of it. In this

subsection we show how to obtain an optimal index assignment by reducing it to a “flow” problem.

Create a new graph, H , from the given graph, G , by “splitting” each vertex of G into two. For each vertex i in G , there are two vertices, x_i and y_i , in H , with an edge from x_i to y_i . Corresponding to every edge ij in G , draw an edge $y_i x_j$ in H . Thus each vertex in G (and each edge in G) is associated with a unique edge in H . Now add an artificial “source” node to H from which there is an edge to each of its nodes that has no predecessors. Similarly, we can introduce a “sink” node with an arc to it from each node with no successors in the graph. Every node in the graph is reachable from the source node, and every node can reach the sink node. The mutual reachability of any pair of nodes in the graph is not affected by the introduction of such a source and sink.

Consider a physical chain G that begins at a node i and ends at a node j . There corresponds to this chain, a chain in H that begins at the source node and goes through x_i, y_i , etc., in succession, ending via x_j, y_j in the sink node. Corresponding to every intermediate node, k , that the chain passes through in G , the corresponding chain in H passes through node x_k, y_k , and includes arc $x_k y_k$. Conversely, every chain from source to sink in H corresponds to a physical chain in G .

Assign a nonnegative integer flow to each arc in the new graph, such that the flow on the $x_i y_i$ edges is strictly positive for all i , the flow into each node is equal to the flow out of that node (no “accumulation” occurs at any node along the way), and the total flow from the source to the sink is minimized. This problem is a standard *min-flow* problem and can be solved for an acyclic graph by well-known techniques such as in [15, 24, 25, 29].

The solution to the *min-flow* problem is a set of values for flows, one value for each edge in H . The total flow out of the source, or into the sink, is the number of chains required to cover the given graph. The chains themselves may not be determined uniquely, but finding at least one chain cover is trivial. Start from the source and follow any edge with a positive flow on it, decrementing the flow by one as the edge is traversed. When the sink is eventually reached, the edges traversed constitute a physical chain. In other words, a chain is found by conceptually tagging a particular unit of flow and following it from source to sink. Then this unit of flow is removed from further consideration, by decrementing the flow on all the edges traversed by this unit, and the next unit of flow tagged from source to sink to find the next chain.

The above procedure gives a set of physical chains that cover the nodes in the graph G , with some nodes possibly lying on more than one chain. Such nodes are arbitrarily assigned to one of the chains and deleted from the rest, thereby obtaining an index assignment. The total effort required is $O(n^3)$ for a graph with n nodes.

Example 3. Minimum Chain Cover Obtained as a Flow. Consider the graph G given in Figure 2a. Each of its nodes can be split and new source and sink nodes added to create the graph H of Figure 2b. Two units of flow are required from source to sink if each $x_i y_i$ edge is to have at least one unit through it. The solution to the *min-flow* problem is shown in Figure 2b. Tracing the flow in the solution, we find that one unit flows from the source through x_A, y_A, x_D, y_D to the sink. The other unit flows from the source through $x_A, y_A, x_C, y_C, x_B, y_B$, to the

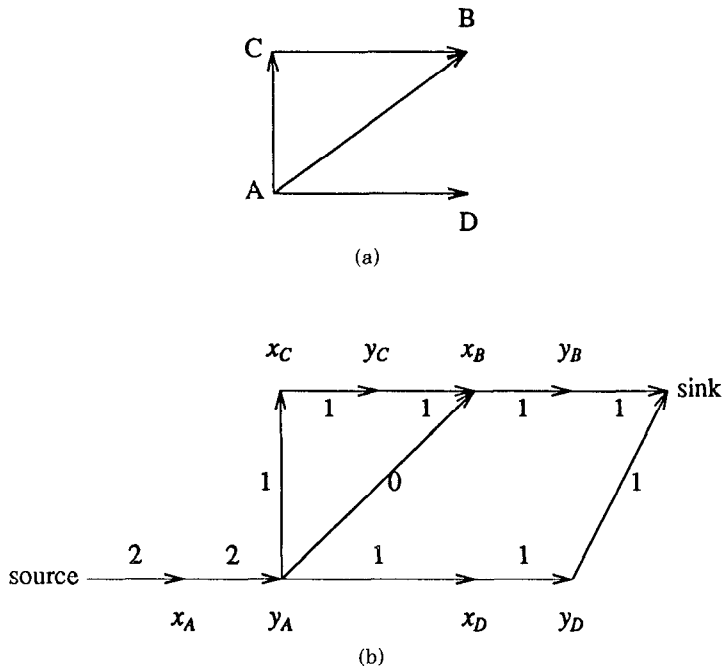


Fig. 2. (a) A graph G , and (b) its derivative H , after nodes have been split.

sink. The corresponding chains are A - D , and A - C - B . Node A occurs on both chains. It can be deleted from one of them, say the first, to obtain the chains A - C - B and D , which can be used to assign indices to the nodes.

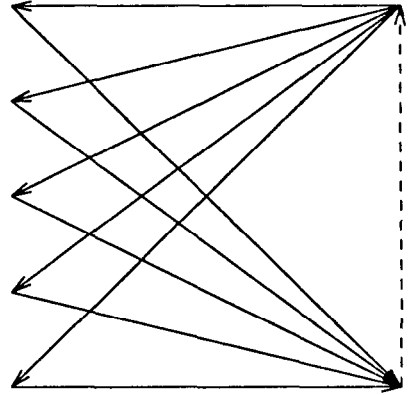
3.3 Graphs with Cycles

The width theorem and the optimal index assignment procedure above apply only to acyclic graphs. In the case of graphs with cycles, determining whether a single chain (which does not intersect itself) can cover all the nodes in the graph is known as the Hamiltonian path problem and is np -complete. Since a special case is np -complete, finding a minimum chain cover for a graph with cycles is np -hard.

Fortunately, every cyclic graph can be reduced to a smaller acyclic graph while maintaining all the reachability information. This is because all nodes on a cycle are mutually reachable, and hence have identical reachability properties, in that they can each reach the same set of nodes, and can each be reached by the same set of nodes. Therefore, one need store only one common (compressed) successor or predecessor list for all of these nodes.

Formally, one can find all the *strongly connected components* in the given graph, using, say, Tarjan's algorithm [32]. Then each strongly connected component can be collapsed into a single node resulting in an acyclic graph. The transitive closure of this acyclic graph can be computed and compressed using techniques such as those suggested above.

Fig. 3. This graph is a single strongly connected component if the dashed edge is present, and each node is a distinct strongly connected component if the dashed edge is absent.



Now each index pair is associated not with an individual node but rather with a set of nodes that all belong to a single strongly connected component. Once more, the indices must be stored in such a way that it is possible from a specified node to obtain its index pair, or from a specified index pair to obtain all the nodes that correspond to it, if queries of the nature of “find all successors” are expected.

A drawback of organizing a graph into strongly connected components is that the constitution of the components is “brittle” in the sense that the removal of a single edge could cause drastic changes in the structure of the strongly connected components. See Figure 3. Consequently incremental update is difficult for a transitive closure compressed in this manner. We propose an alternative below.

If chains are permitted to intersect themselves, that is, if a node is permitted to appear more than once on a chain, then a minimum chain cover can be found for a cyclic graph in polynomial time. ($(O(s^3) + O(E))$ effort for a cyclic graph with E edges and s strongly connected components). If a node appears more than once on a chain, then only one of the occurrences is used to determine its position, say the first occurrence. Whether chains intersect themselves or not, it is possible for a node in a graph with cycles to reach nodes on the same chain that precede it.

Every strongly connected component can be covered by a single chain that possibly intersects itself. A minimum chain cover for a graph with cycles can be obtained from the minimum chain cover for the equivalent acyclic graph with strongly connected components collapsed as follows. In the graph with cycles, mark the edges between strongly connected components that constituted a chain in the corresponding acyclic graph with components collapsed. For each strongly connected component that lies on the chain there will be a marked edge to a node in the component and a marked edge out of a node (which may be the same or a different node) in the component. Find a path from the former to the latter covering all the nodes in the given strongly connected component. Include all edges of this path in the chain. Proceed similarly for the other chains.

Example 4. Index Assignment for a Graph with Cycles. Consider the graph shown in Figure 4a, which is the graph of Figure 1a with one of the edges

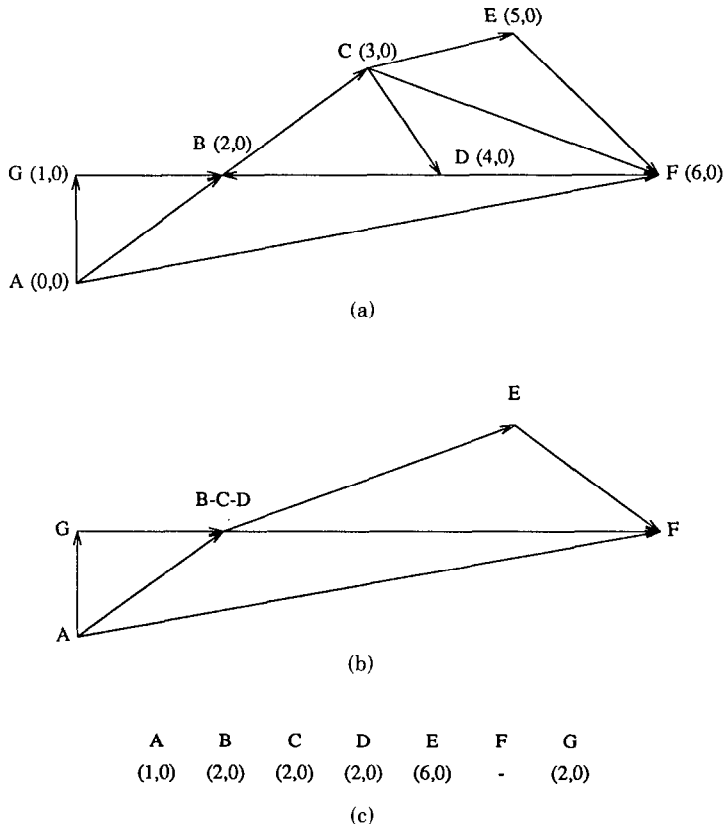


Fig. 4. (a) A graph with its index assignment shown, (b) its reduction with strongly connected components collapsed, and (c) the compressed successor lists for the nodes in this graph.

reversed. The nodes B, C, D form a strongly connected component. From Figure 4b, it is evident that a single chain, A-G-(B, C, D)-E-F, can cover all the nodes in the graph. The strongly connected component must be entered by this chain at B and must be exited at C. Therefore, the chain in Figure 4a is A-G-B-C-D-B-C-E-F. The second appearance on the chain is ignored for nodes B and C. The resulting index assignment is shown in Figure 4a, and the compressed transitive closure in Figure 4c. Observe that node D has indices (4, 0), but has an entry of (2, 0) in its compressed successor list. Also, since node B has a nontrivial path to itself, its successor list has an entry (2, 0) rather than (3, 0).

3.4 Heuristics for Index Assignment

We have so far shown how to generate an index assignment in polynomial time that is optimal in the sense that the number of chains is minimized. However, for large databases, even such "simple" procedures may take too long. (The effort involved is of the same order of magnitude as computing the transitive closure

of the relation). Simpler (linear time) heuristics may be preferred that produce solutions close to optimal.

Moreover, the “optimal” solution obtained above merely minimizes the number of chains, and hence establishes a small upper bound on the amount of storage required for the compressed successor lists. (There are no more entries for the compressed successor list of each node than there are chains in the graph). Our real interest is not to minimize the number of chains, but to minimize the amount of storage required, which is the sum over all nodes of the number of chains that are accessible from that node. Since not every chain will be accessible from each node, storage is not necessarily minimized if the number of chains is minimized.

In the light of the foregoing, one may prefer a quick and dirty index assignment. Several simple, but suboptimal, ways to determine a reasonable set of indices for the nodes are described below. These techniques apply equally to graphs with and without cycles. Each procedure is proved correct by showing that it assigns a unique index pair to each node, and that the index assignment obtained satisfies the *Indexing Rule*.

3.4.1 Chain-Order Heuristic

Procedure 1

Mark all nodes **unused**.

Set $k = 0$.

While there exist **unused** nodes,

 Pick an **unused** node and call it the **current** node.

 Assign the **current** node indices $(0, k)$, and mark it **used**.

 Set $i = 1$.

 While there exist **unused** successors of the **current** node,

 If there is an **unused** immediate successor of the **current** node,

 Call one such successor the **current** node.

 Else pick any **unused** successor of the **current** node and call it the **current** node.

 Assign the **current** node indices (i, k) and mark it **used**.

 Increment i .

 Increment k .

PROOF. Since i is incremented after each index assignment and since k is incremented whenever the value of i is reduced, each node is assigned a unique index pair. The node marked (i, k) in the procedure is a successor of the node marked $(i - 1, k)$ for all $i > 0$. Therefore, by transitivity of the *successor* relationship, node (i, k) is a successor of node (j, k) for all $0 \leq j < i$. No node is assigned an index (i, k) with $i < 0$, in Procedure 1. Hence, node (i, k) is a successor of node (j, k) for all $j < i$, and the index assignment satisfies the *indexing rule*. $\square$

The basic idea in the chain-order heuristic is to start a chain and to keep on extending it to the extent possible. The chain ends when no more successors can be found. After all the chains have been formed by the above procedure one could concatenate any pair of chains in which the last node of one chain can reach the first node of the other, thereby reducing the number of chains. This post-processing step is known as *chain concatenation*. It can be applied irrespective of the techniques used to derive the chains.

Assuming that the transitive closure of the graph has been precomputed, the effort to execute Procedure 1 can be calculated as follows, for a graph with n nodes and E edges. $O(n)$ effort to mark each node **unused**. Thereafter, for each node, up to n successors may have to be examined to find one that is **unused** (or to determine that there are none that are **unused**), giving a total effort of $O(n^2)$. However, the algorithm can be simplified by extending a chain with **unused** immediate successors only, not extending it with more distant successors if all immediate successors are used. With this simplification, Procedure 1 requires $O(E + n)$ effort and does not need precomputation of the closure. (The reason for the $O(E + n)$ effort is that for each node, each immediate successor is examined once, to check whether it is **used**. That is to say, a constant amount of work is done once for each edge in the graph. The $+n$ is required since each node is processed even if it has no successors). Shorter chains may be produced due to the simplification, but are likely to get combined in the course of chain concatenation.

Depending on the manner in which the original graph and its transitive closure have been stored, it may or may not be easy to use predecessors or immediate predecessors to extend a chain “backwards”, the way successors are used in Procedure 1 to extend a chain “forward”. Experimental results, not reported in this paper, indicated that the use of predecessors to extend chains was not of value if chain concatenation was used. As such, this option is not explored further.

3.4.2 Node-Order Heuristic.¹ The chain-order heuristic discussed above developed the chains one by one, and in the process examined nodes as required, often examining each node several times. Since the number of nodes is usually much larger than the number of chains, for most databases, such repeated node examination could lead to excessive paging. The node-order heuristic examines each node exactly once, and at that time assigns it to a chain. Several chains are constructed in parallel, and reference is often made to determine the current head and tail of each chain. If the number of chains is small enough, this data may be cached in memory.

Procedure 2

```

 $k = 0$ 
For each node  $a$ 
  If  $a$  is an immediate successor of the last node in some chain,  $(i, j)$ ,
    Add  $a$  as the new last node of this chain, with indices  $(i + 1, j)$ ,
  Else
    If  $a$  is an immediate predecessor of the first node in some chain,  $(i, j)$ ,
      Add  $a$  as the new first node of this chain, with indices  $(i - 1, j)$ 
    Else
      Assign  $a$  the indices  $(0, k)$ 
       $k = k + 1$ 
/*Start a new chain, with  $a$  the only (first and last) node in it */

```

PROOF. Consider a chain j . The last node in this chain, to begin with, is the only node in this chain, $(0, j)$. When a new node is added to the chain as the last

¹ This heuristic was suggested by Rakesh Agrawal.

node, it has indices $(1, j)$, with the first index being the greatest among the two nodes on chain j . If at some later stage in the execution of the procedure, (i, j) is the last node of chain j , and a new last node is added, it has indices $(i + 1, j)$. If i was the greatest value for the first index among nodes on chain j , then $i + 1$ is the new greatest value. Therefore, by induction on i , if (i, j) is the last node in chain j , for every node (k, j) on chain j , $k \leq i$. Similarly, if (i, j) is the first node in chain j , for every node (k, j) on chain j , $k \geq i$.

A node is assigned an index $(i + 1, j)$ where (i, j) is the last node in chain j , or an index $(i - 1, j)$ where (i, j) is the first node in chain j . In either case, the indices assigned must be unique since there can be no node with index $(i + 1, j)$ in chain j if (i, j) is the last node, or with index $(i - 1, j)$ if (i, j) is the first node. Moreover, if (i, j) is the last node of a chain, every node (k, j) on the chain has $k \leq i$, implying $k < i + 1$. But if the last node of a chain can reach the new node, then, by transitivity, so can every node on the chain. Similar arguments apply to the first node of the chain. Therefore the *indexing rule* is satisfied. $\square$

A consequence of Procedure 2 is that nodes on a chain could have positive or negative values for their first index. An extra pass can be made to normalize the values to be positive and start from 0 (or 1), if desired.

One could add node a to a chain if it is a successor (not necessarily immediate) of the last node in the chain, or a predecessor of the first node in the chain. Such a heuristic could improve the compression, by increasing the chance of finding a chain to which a given node may be added, but could also worsen the compression, especially for acyclic graphs, since the chains become sparser as a result of not requiring immediacy. This difficulty can be ameliorated by first looking for immediate successors or predecessors that are end-points of some chain, and upon failing to find them, trying other successors and predecessors. The heuristic then is:

Procedure 3

$k = 0$

For each node a

 If a is an immediate successor of the last node in some chain, (i, j) ,

 Add a as the new last node of this chain, with indices $(i + 1, j)$

 Else

 If a is an immediate predecessor of the first node in some chain (i, j) ,

 And a as the new first node of this chain, with indices $(i - 1, j)$

 Else

 If a is a successor of the last node in some chain, (i, j) ,

 Add a as the new last node of this chain, with indices $(i + 1, j)$

 Else

 If a is a predecessor of the first node in some chain, (i, j) ,

 Add a as the new first node of this chain, with indices $(i - 1, j)$

 Else

 Assign a the indices $(0, k)$ /*Start a new chain*/

$k = k + 1$

3.4.3 The Benefits of Topological Sort. In both of the above families of heuristics the compression achieved is greatly improved if the nodes are arranged in a topologically sorted order. (The nodes of a graph are said to be numbered in topologically sorted order if every edge is from a lower numbered node to a higher numbered node. Such a numbering is possible for all directed acyclic graphs and

only for directed acyclic graphs). The reason is that, given a topologically sorted order, we know that all successors of some node under consideration are numbered higher than the node itself, and that all chains consist of nodes arranged in ascending order.

In the chain order heuristic, one can pick the lowest numbered successor of the current last node to extend a chain. Since this successor has the most nodes after it in the topological sort, it is most likely to result in a large growth for the chain. In the node-order heuristic, considering nodes in topologically sorted order provides the same benefit. In addition, new nodes are always added after the last node of some chain, thereby reducing by about half the effort to obtain the index assignment, and thereby also ensuring that all indices assigned are nonnegative.

If we know that a graph is acyclic, often the corresponding database may already be given in a topologically sorted order. If the graph has cycles in it, Tarjan's algorithm for strongly connected components gives the components in topologically sorted order. Even if the graph is not given in topologically sorted order, it may be worthwhile to sort it topologically to improve the compression obtained, especially if few updates are expected. See Section 5 for experimental verification.

4. CHAIN REDUCTION

It is the maximum width of a graph that determines the number of chains that are required to cover all the nodes and, indirectly, the storage required for the compressed closure. However, most graphs are not uniformly "wide". In consequence, it is often the case that a few long chains cover most of the graph and many short chains are required to cover a small fraction of the nodes that are left over. In this section, we discuss the possibility of retaining only the few "long" chains in the compressed closure, and storing the remaining nodes separately in a way that minimizes the storage required. This process is called *chain reduction*.

Let us suppose that a small set of such long chains has been identified for a graph, and that the other (small) chains have been deleted. Let the nodes that lie on the chains be called *chain nodes*; let the other nodes be *off-chain nodes*. Most nodes in the graph should be chain nodes, so that good compression is achieved at each chain node for its chain node successors, but a lot of storage is being required for its off-chain successors. We propose now that we not store the off-chain successors of chain nodes at all: instead we shall store the same chain nodes as *predecessors* of the off-chain nodes. Now at each off-chain node we have many predecessors that all lie on the same chain, and are therefore subject to compression on account of the *Indexing rule*. Of the predecessors on a chain, only the last need explicitly be retained in the compressed predecessor list. Earlier nodes on the chain can be inferred to be predecessors by transitivity.

Let there be n_i chain nodes and n_o off-chain nodes. $n = n_i + n_o$. $n_i \gg n_o$. Let there be c_i chains retained and c_o chains deleted. $c = c_i + c_o$. $c_o \gg c_i$. Chain reduction substitutes $O(n_o c_i)$ entries in the compressed predecessor lists of off-chain nodes for $O(n_i c_o)$ entries in the compressed successor lists of chain nodes.

At each off-chain node, we have also to keep track of its off-chain successors. We could do this individually, resulting effectively in a full transitive closure of

the off-chain nodes. ($O(n_0^2)$ storage required rather than the $O(n_0 c_0)$ required without chain reduction). The size of this transitive closure will be small if the number of off-chain nodes is very small. The penalty for maintaining this closure is also small if the sizes of the deleted chains are small.

If a small join is permitted at query time instead of just a simple look-up, storage can be reduced considerably, by using the chain structure. In the successor list of an off-chain node, a , one need store only those off-chain nodes that do not have a chain node predecessor that is also a chain node successor of a .

We can now write the compression procedure including chain reduction as follows:

- (1) Form chains and assign indices to all the nodes using any of the procedures given in Section 3.
- (2) Remove indices from, and mark, nodes that lie on all chains except the few longest. An empirical threshold could be set for the smallest length of chain that is acceptable, or for the maximum number of chains that should be retained.
- (3) For each chain node, compress the successor list by deleting all dominated chain nodes and deleting all off-chain nodes in the successor list. When deleting an off-chain node a from the successor list of a chain node b , add b to the predecessor list of a .
- (4) Compress the predecessor list of each off-chain node by deleting inverse-dominated nodes. (The notion of inverse-domination in a predecessor list is analogous to the notion of domination for successors. A node with index (i, j) *inverse-dominates* a node with index (k, j) iff $i > k$, since every node that can be reached by node (i, j) can also be reached by node (k, j) . u dominates v iff v inverse-dominates u).
- (5) Compress the successor list of each off-chain node by deleting dominated chain nodes.
- (6) Compress the successor list of each off-chain node a by deleting off-chain node b from the successor list if b has a chain node predecessor that is a chain node successor of a .
(This reachability check can be performed by simply considering each chain in turn for the pair of nodes in question, and comparing the indices of the respective successor and predecessor. The only information required for this step is the successor list of a and the predecessor list of b for each pair of nodes a, b).

The procedure for determining whether a given destination node d is reachable from a given source s is as follows:

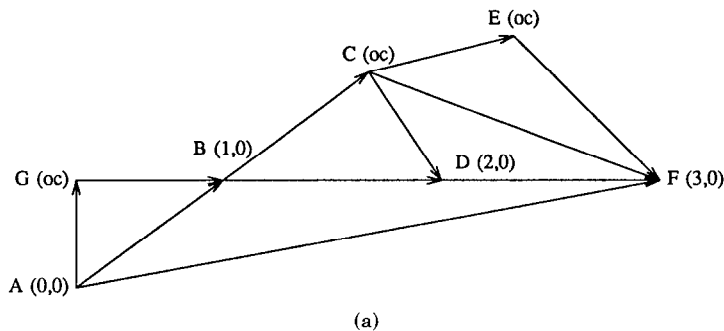
- (1) If d is a chain node, there exists a path from s to d iff:
 - (a) d is the compressed successor list of s , OR
 - (b) there is a node v in the compressed successor list of s such that v dominates d . (This is the regular look-up rule).
- (2) If d is an off-chain node, but s is a chain-node, there exists a path from s to d iff:
 - (a) s is in the compressed predecessor list of d , OR

- (b) there is a node v in the compressed predecessor list of d such that v inverse-dominates s , (that is, v has a higher value for the first index than s).
- (3) If both s and d are off-chain nodes, there exists a path from s to d iff:
 - (a) d is the compressed successor list of s , OR
 - (b) there exists a chain-node, v , in the compressed successor list of s that is the same as or dominates a chain-node, w , in the compressed predecessor list of d . (This condition can be checked by performing a “composition”² on the chain number of the successor list of the former and the predecessor list of the latter, with both lists interpreted as relations with two fields for the two indices. In the result of the join, if there exists a tuple in which the value of the first field is less than or equal to the value of the second, then this condition is satisfied).

Correctness. Correctness of this procedure is demonstrated by establishing the “iff” implication both ways for each case individually.

- (1) If d is a chain-node in the successor list of s , then it is either retained in the compressed successor list of s , or is deleted during steps 3 or 5. If it is deleted, then there exists a node v in the compressed successor list of v that dominates d . Thus, if d is a successor of s , then one of the conditions stated in the procedure must be satisfied. The converse follows directly from the indexing rule.
- (2) d is an off-chain node, but s is a chain node. d cannot be in the compressed successor list of s , since it would have been deleted in step 3. However, if d was in the successor list of s , and was deleted in step 3, then s has been included in the predecessor list of d . Thereafter, s may have been retained in the compressed predecessor list of d , or it may have been deleted in Step 4, due to the existence of a node v in the compressed predecessor list of d that inverse-dominates s . Thus, if d is a successor of s , then one of the conditions stated in the procedure must be satisfied. The converse follows from the indexing rule: If v inverse-dominates s and there is a path from v to d , then there is a path from s to d by transitivity.
- (3) Both s and d are off-chain nodes. If d is a successor of s , either d is retained in the compressed successor list of s , or d is deleted in Step 6 of the compression procedure. If d is deleted, there exists a chain-node u that is a successor of s and a predecessor of d . Therefore, the compressed successor list of s has a node v that is either the same as u or dominates u , and the compressed predecessor list of d has a node w that is either u or inverse-dominates u . Nodes v and w lie on the same chain, and either $v = w$, or v dominates w . Hence if d is a successor of s , one of the two conditions stated must be satisfied. The converse is easily established by the indexing rule and transitivity.

² A composition of two relations is their natural join followed by a projection of the join field. A tuple $\langle a, b \rangle$ in the first relation composes with a tuple $\langle b, c \rangle$ in the second relation to produce a result tuple $\langle a, c \rangle$.



Node	A	B	C	D	E	F	G
Chain Successor	(1,0)	(2,0)	(2,0)	(3,0)	(3,0)	-	(1,0)
Chain Predecessor			(1,0)		(1,0)		(0,0)
Off-Chain Successor			E		-		-

(b)

Fig. 5. (a) A directed acyclic graph with its index assignment shown after chain reduction, and (b) the compressed successor and predecessor lists for the nodes in this graph.

Example 5. Compressed with Chain Reduction. Consider once again the graph of Figure 1 with the same chains as before. This time, let us delete the shorter chain G-C-E and retain only the longer chain A-B-D-F. The resulting index assignment is shown in Figure 5a, where (oc) refers to an off-chain node. The compressed (chain) successor lists are obtained as before. The only chain node that can reach G is A, with index (0, 0); so (0, 0) is placed in the compressed chain predecessor list of G. Both A and B can reach off-chain nodes C and E. Only the latter of the two, B, is kept, as index (1, 0), in the compressed predecessor list of these nodes. Finally, in forming the off-chain successor lists, we note that G can reach C, but since G can reach B and B can reach C, we do not store C as an off-chain successor of G. Similarly, we do not store E as an off-chain successor of G. We do, however, store E as an off-chain successor of C since no chain nodes intervene. The resulting compressed storage structure is shown in Figure 5b.

Given the compressed store, let us see how a few different queries would be answered. A path is found from A to a chain node D since D is (2, 0) and (1, 0) is on A's successor list. A path is found from chain node A to off-chain node E since A is (0, 0) and (1, 0) is on E's predecessor list. A path is found from an off-chain node G to off-chain node E since chain node B is reachable from G and can reach E. A path is found from off-chain node C to off-chain node E by direct look-up.

One drawback of chain reduction that should be noted is that now not all the successors of a node can be read off its successor list. Thus, whereas reachability queries between two specified points can be answered quickly, each off-chain

node has to be considered individually if one is asked to list all the successors of a specified (chain or off-chain) node.

Post-Reduction Compression. Recall that in a transitive closure compressed with chain reduction, a join has to be performed to determine whether a path exists between an off-chain node and another off-chain node. Reachability between all other pairs of nodes can be determined by a look-up (and comparison). Further compression is possible if one is willing to perform a join rather than do a straight look-up. This additional compression is applicable to the chain node predecessor list and the chain node successor list of every off-chain node.

The compression procedure is simple. Once the reduced compressed storage structure has been formed, examine the chain node successor list of each off-chain node in turn. If any of the nodes in the list can be reached by another, delete the node that can be reached from this list. In a similar fashion, compress the predecessor list as well.

Now, determining whether a node s can reach another d is done as follows:

- (1) If both s and d are chain nodes, there exists a path from s to d iff:
 - (a) d is in the compressed successor list of s , OR
 - (b) there is a node v in the compressed successor list of s such that v dominates d . (This is the regular look-up rule).
- (2) If s is an off-chain node, but d is a chain node, there exists a path from s to d iff:
 - (a) d is the compressed successor list of s , OR
 - (b) there is a node v in the compressed successor list of s such that v dominates d , OR
 - (c) there is a node v in the compressed successor list of s such that there is a path from v to d . (Both v and d are chain nodes, so condition 1 applies).
- (3) If d is an off-chain node, but s is a chain node, there exists a path from s to d iff:
 - (a) s is in the compressed predecessor list of d , OR
 - (b) there is a node v in the compressed predecessor list of d such that v inverse-dominates s , OR
 - (c) there is a node v in the compressed predecessor list of d such that there is a path from s to v . (The existence of this path from one chain node to another can be checked by applying condition 1).
- (4) If both s and d are off-chain nodes, there exists a path from s to d iff:
 - (a) d is in the compressed successor list of s , OR
 - (b) there exists a chain-node, v , in the compressed successor list of s that has a path to d . (The existence of a path from v to d can be checked by applying condition 3).

PROOF. Condition 1 is unchanged from the simple reduction discussed above. Correctness is established in exactly the same way. The *if* part for conditions 2, 3, and 4 is easy to establish. The *only if* part is discussed below for each of these three cases.

If an off-chain node s has a path to a chain node d , then either one of 2a and 2b is satisfied, as in the case of simple chain reduction, or a node that would

have caused one of 2a and 2b to be satisfied has been deleted during post-reduction compression. But such a deletion can occur only if there is another node retained in the compressed successor list of s that can reach the deleted node, and hence d . In other words, condition 2c must be satisfied.

If a chain node s has a path to an off-chain node d , then either one of 3a and 3b is satisfied, as in the case of simple chain reduction, or a node that would have caused one of 3a and 3b to be satisfied has been deleted during post-reduction compression. But such a deletion can occur in the compressed predecessor list of d only if there is a node retained that can be reached from the deleted node, and hence s . That is to say, condition 3c must be satisfied.

If an off-chain node s has a path to an off-chain node d , then either d is retained in the compressed successor list of s , so that 4a is satisfied, or d is deleted from the successor list of s during one of the compression stages. But d is deleted from the successor list of s only if there is a v in the compressed successor list of s such that v can reach d , that is, only if 4b is satisfied. $\square$

This additional compression step only reduces the amount of storage required for the chain successors and chain predecessors of the off-chain nodes. Since the total storage is dominated by the other two terms, the additional compression achieved is slight. While no additional work is involved if condition 1 is applicable, a join is now required for each of conditions 2 and 3, and two joins are required (in place of 1) for condition 4.

5. PERFORMANCE EVALUATION

In this section we present the results of several simulation experiments performed to evaluate the compression techniques presented in this paper.

5.1 The Experimental Set-Up

Since the publication of [10], the Wisconsin Benchmarks have become the de facto standard for evaluating the performance of database systems and algorithms. However, these benchmarks are not useful for evaluating recursive queries. A methodology for evaluating recursive queries has been proposed in [7]. This proposal is good for analysis, but has the limitation that it permits only a small restricted set of underlying graphs for the base relation.

The main merit of synthetic databases, such as those used in the Wisconsin Benchmarks, is the ability to specify the selectivity for a relational operation. As a consequence, one can exercise control over the size of the initial and result relations by modifying this selectivity. Following similar arguments, benchmarks were proposed in [2] that would be suitable for evaluating the performance of transitive closure and other recursive algorithms on relational databases. We identify three parameters of a graph as important³: the *number of nodes*, the average *degree* of a node, and the average *depth* of the graph. In a directed graph,

³ In [2], an additional parameter, *locality*, was considered rather than *depth*, for generating synthetic graphs. In a graph with high locality, most arcs from a node connect nearby nodes, with a few arcs that connect far-away nodes. Locality was modeled in terms of the length of an arc, where the length of an arc was taken to be the absolute difference of its source node number and destination node number. We do not consider locality in this study since the locality of a graph depends on the node numbering.

one must be concerned with both the out-degree and the in-degree of a node. However, each edge has a source node and a destination node; due to the edge, the source node has its out-degree increased by one, and the destination node has its in-degree increased by one. Therefore, over all the nodes in the graph, the average in-degree is equal to the average out-degree, and only one need be specified. We use datasets with only the out-degree specified. The out-degree of each node is obtained from a uniform distribution between zero and twice the specified mean. The graph generation procedure consists of first determining for each node its out-degree (by sampling a distribution), and then for each outgoing arc from this node, determining randomly the destination node (by sampling another distribution).

In the experiments performed, we considered both graphs with cycles and acyclic graphs. Cyclic graphs arise naturally in many applications such as flights database, a database of inter-city bus routes, or a communication network. Acyclic graphs arise in applications like project management, part hierarchies, genealogical hierarchies, etc.

The third parameter, *depth*, applied only to acyclic graphs. Just as in the case of out-degree, we used an average depth rather than the maximum. Thus for a balanced binary tree, the root is at depth zero, its immediate children are at depth one, and so forth. If the leaves are at depth d , the average depth works out to $d - 1 + d + 1/(2^{d+1} - 1)$, which is just a little over $d - 1$. For unbalanced trees, not so many leaves are the maximum depth, so the average depth is likely to be much smaller than the maximum. For arbitrary acyclic graphs, every node with no predecessors is assumed to be at depth zero, each of its immediate successors at depth one, and so forth, each node being assigned a depth equal to one greater than the immediate predecessor with the least depth.

The depth of an acyclic graph can be as low as zero for a graph with no edges, or almost zero for a star graph with all edges directed inwards. The depth of an acyclic graph with n nodes can be as large as $(n - 1)/2$, in the case of a single long chain. Any value in between is possible. A graph with a large depth is likely to be "tall and skinny" while a graph with a small depth is likely to be "short and squat." Unfortunately, however, the depth of a graph is not orthogonal to the number of nodes in the graph and its degree.

Number of nodes, average degree, and, where applicable, depth, are properties easy to determine for a given initial relation. Between them they characterize a graph, and can be used to guess properties of the resulting transitive closure. For example, graphs with more nodes tend to have larger closures as do graphs with a higher degree. However, there is no predicting formula, and counter-examples exist. For example, a low degree graph, such as a single directed cycle, could have every node able to reach every other, a characteristic one would normally expect only in graphs with a much higher degree.

The total storage required was simply computed as the sum of the sizes of the successor lists at each node, size being measured in number of entries. Thus, the storage for the original database is the sum of sizes of the immediate successor lists at each node, which is equal to the number of tuples in the database, or the number of arcs in the graph. Similarly, the storage for the uncompressed transitive closure is the number of tuples in the transitive closure database or the number of arcs in the transitive closure graph.

In the case of the compressed transitive closure, for each entry in the successor list, one has to store a pair of indices rather than a single node identifier. Therefore, it may be appropriate to multiply by two all the storage sizes mentioned below for the compressed closure. We chose to present the results in terms of the number of entries, rather than in terms of number of bytes or words, because exactly what the penalty will be for the two-index entries required by the compression scheme is hard to determine. For example, if the nodes were originally identified by strings, an extra integer may be a small penalty. On the other hand, the penalty will increase, and may even become greater than two, if chain reduction is used. The point is that the penalty can be controlled by intelligently managing the data structures used for storing these lists.

For all the graphs given below we ran all three heuristic procedures described in Section 3, both with and without chain concatenation, and both with and without the use of topological sort, giving rise to a total of twelve different index assignment heuristics. For each of these twelve index assignments for each graph, the compression was measured, without chain reduction, with chain reduction, and with chain reduction and post-reduction compression, giving a total of 36 measurements for each graph. In all the figures given below, the storage requirement for the compressed closure is given as a fraction of the storage requirement for the full transitive closure. To minimize clutter, only an “interesting” subset of these 36 measurements is plotted in each figure.

With the use of topological sort to order the nodes for index assignment, all three heuristics did about equally well, and chain concatenation was of little additional value. As such, we plot the measurements only for Procedure 1 after topological sort, without chain concatenation. Other index assignment techniques will do approximately as well given the topological ordering. Procedure 3, by forming chains of nodes that are not immediate successors, renders chain concatenation superfluous. As such, we need consider Procedure 3 only without chain concatenation. Thus, the graphs below have six curves representing twelve measurements.

5.2 Acyclic Graphs

Figures 6a–c show the size of the compressed transitive closure for different index assignments applied to a directed acyclic graph, without chain reduction. In Figure 6a, we see that for all the procedures, the relative size of the compressed closure falls gradually as the number of nodes increases. The absolute size of the compressed closure obviously increases, but the size of the full transitive closure increases even faster.

In Figure 6b, we see a dramatic reduction in the relative size of the compressed closure as the degree of the graph is increased. To understand the source of this large change consider Figure 7. In this figure, the same variation with out-degree is plotted on an absolute scale. Also, more detail is provided by considering a larger graph and a larger range of values for the outdegree. We see that as the degree of the graph is increased, at first the size of the transitive closure shoots up, as new paths are found to nodes that previously were unreachable. (A degree 1 graph is likely to consist of several unconnected components). Later on, further increases in the degree do not increase the size of the closure as much, since most of the nodes are already reachable and addition of more arcs establishes

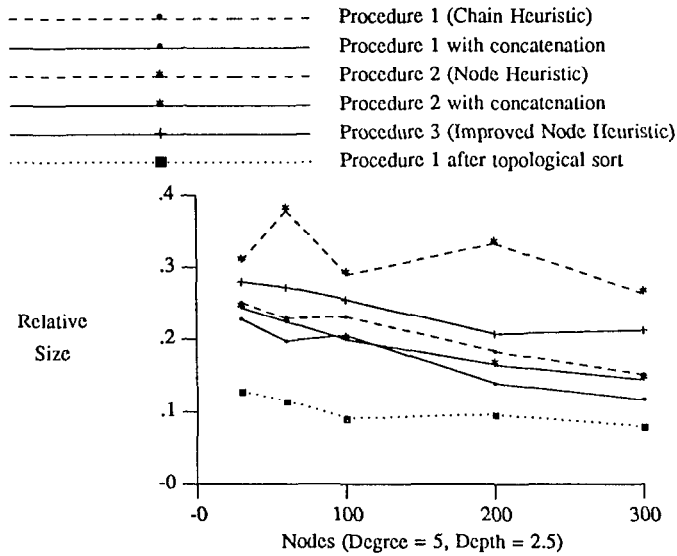


Fig. 6a. Compression of transitive closure on a directed acyclic graph as the number of nodes is varied.

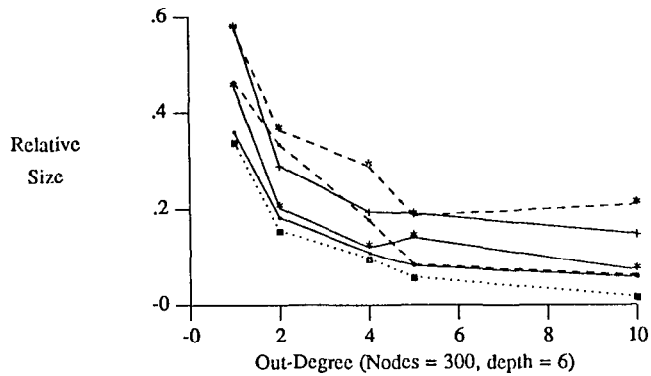


Fig. 6b. Compression of transitive closure on a directed acyclic graph as the out-degree is varied.

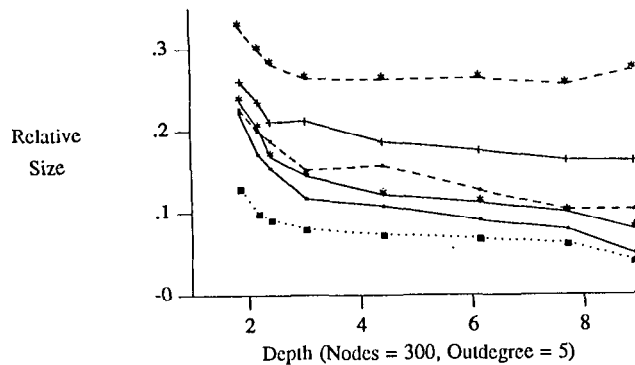


Fig. 6c. Compression of transitive closure on a directed acyclic graph as the depth is varied.

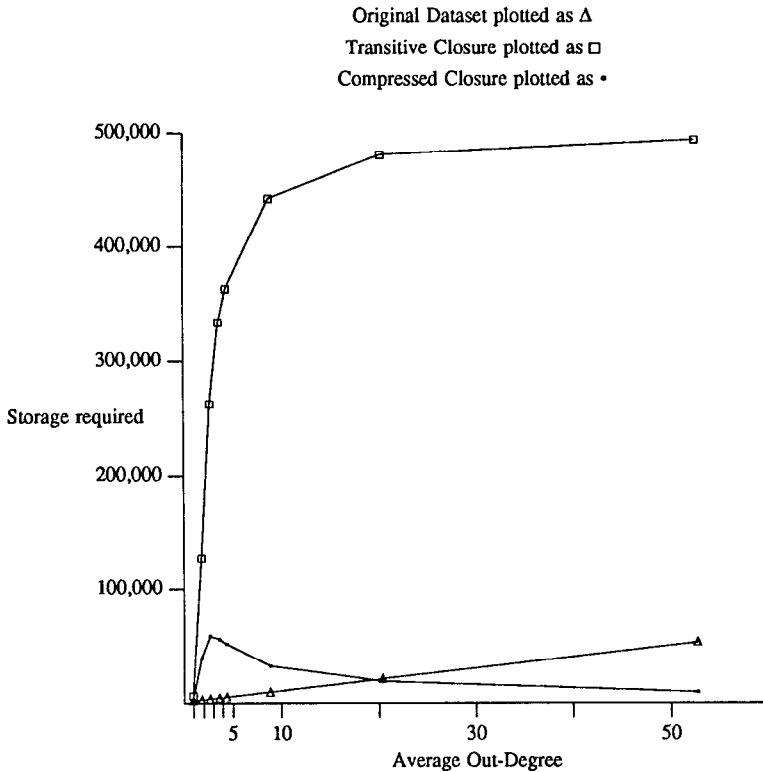


Fig. 7. Compression of transitive closure as degree is varied on a 1000-node directed acyclic graph with depth 10.

more alternate paths to nodes that are already reachable rather than finding new nodes that can now be reached. Witness, for example, the fact that of the 495,000 possible arcs in a 1000 node acyclic graph, 442,000 are already in the closure for a degree of 9 in Figure 7.

The size of the compressed closure also rises rapidly at first, though less so than the size of the full transitive closure. As more and more arcs are added, however, the number of chains required to cover the graph decreases, since it is less likely that there are pairs of nodes neither of which can reach the other. Since the number of chains is decreasing and the number of successors at any node is increasing only slowly, the size of the compressed closure actually begins to decrease as the degree of the graph is increased, and eventually becomes even less than the size of the original graph.

That the compressed closure requires less storage than even the original graph in some cases should not cause any surprise. Remember that in the compressed closure, we wish to store only the reachability information in the underlying graph and do not attempt to maintain any path information. A graph of high degree has many "redundant" arcs whose removal will not affect the transitive closure. The compressed closure gets rid of the extra storage required for these redundant arcs. Of course, the original graph cannot be recovered from the

compressed closure. As an extreme illustration consider a complete graph with n nodes. Such a graph would have nearly n^2 arcs. The closure would be the same as the original graph. To compress the closure a single chain suffices. As such, the compressed successor list of each node has but a single entry. Consequently,⁴ the compressed closure requires only n storage even though the original graph required n^2 .

Figure 6c demonstrates the effect of depth. As expected, graphs with a greater depth compress better than graphs with a smaller depth. Observe though, that graphs with a higher depth are likely to have larger transitive closures than graphs with a low depth. In summary, from Figure 6, we can conclude that graphs with more nodes, larger degree, and greater depth are likely to compress better using our techniques. These also are graphs that are likely to have larger closures.

Returning to Figure 6a and comparing the two versions of each heuristic for index assignment, we find that chain concatenation is of great benefit for Procedures 1 and 2. Procedure 3 does better than Procedure 2 without chain concatenation, as indeed one would expect since the former spends more effort to construct fewer chains than the latter. It is interesting to note, however, that Procedure 3, by forming chains of nodes that are not immediate successors, renders chain concatenation worthless and actually fares worse than Procedure 2 with chain concatenation. Figure 6b and 6c also show the same trends.

All three procedures were applied with and without chain concatenation to the same acyclic graphs after the graphs had been topologically sorted. The resulting compression was more or less the same in all cases and only a single line has been plotted for the compression after topological sort. For example, in the absence of a topological sort, Procedure 1 clearly outperforms Procedure 2. However, the two procedures do about as well on a topologically sorted graph. In addition, the compression achieved is much better than that for either procedure without the topological sort. The benefits of topological sorting are evident.

Figures 8a–c are analogous to Figures 6a–c, and show the same procedures applied to the same set of graphs, except that chain reduction has additionally been applied. The procedure used was to obtain the full chain cover and then to start deleting the chains one by one, beginning with the smallest chains, each time recomputing the amount of storage required. In the beginning, as very small and inefficient chains are removed, there is a big improvement in the storage required. Then, as more and more chains are deleted, the storage for the lists of off-chain successors starts rising, so that after a minimum point is passed, the total storage required begins to increase. Finally, every node is an off-chain node, there are no chains at all, and the storage requirement is that of a full transitive closure. The numbers plotted are for the minimum point for each case. Figure 9 shows this initial improvement in compression as the shorter chains are deleted followed by a worsening as too many chains are deleted. Notice that even with only a single chain considerable compression is achieved. Though this figure is for a particular graph using a particular index assignment heuristic, similar trends were observed for other graphs and other heuristics.

⁴ Note, however, that we are storing only reachability information in our compressed structures. Typically, graphs in which reachability information is interesting will have only a small fraction of all nodes reachable from any node.

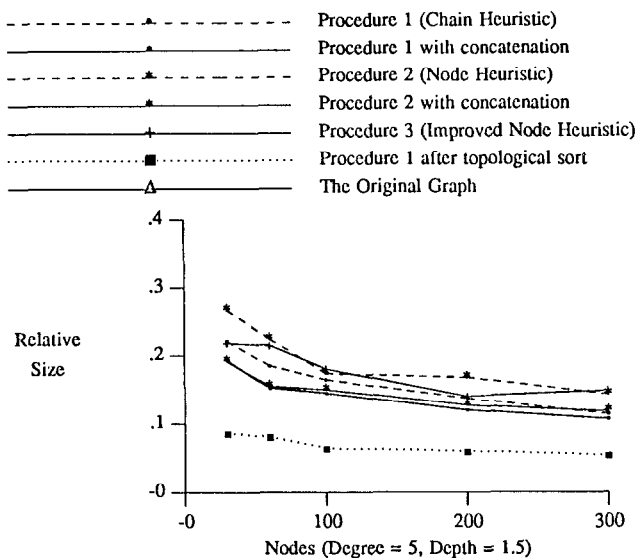


Fig. 8a. Compression, after chain reduction, of the transitive closure on a directed acyclic graph as the number of nodes is varied.

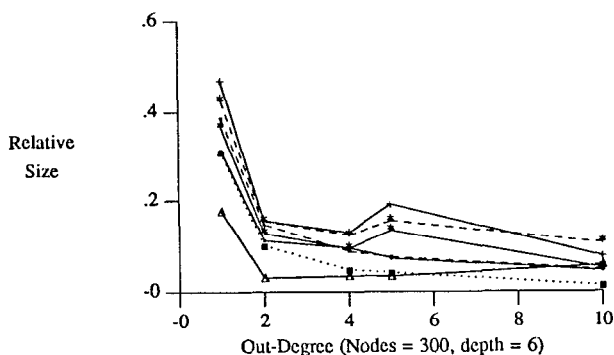


Fig. 8b. Compression, after chain reduction, of the transitive closure on a directed acyclic graph as the out-degree is varied.

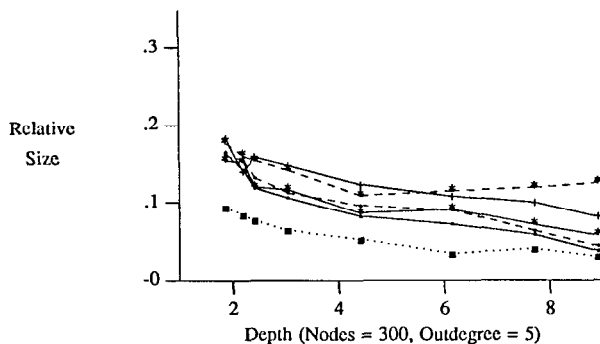


Fig. 8c. Compression, after chain reduction, of the transitive closure on a directed acyclic graph as the depth is varied.

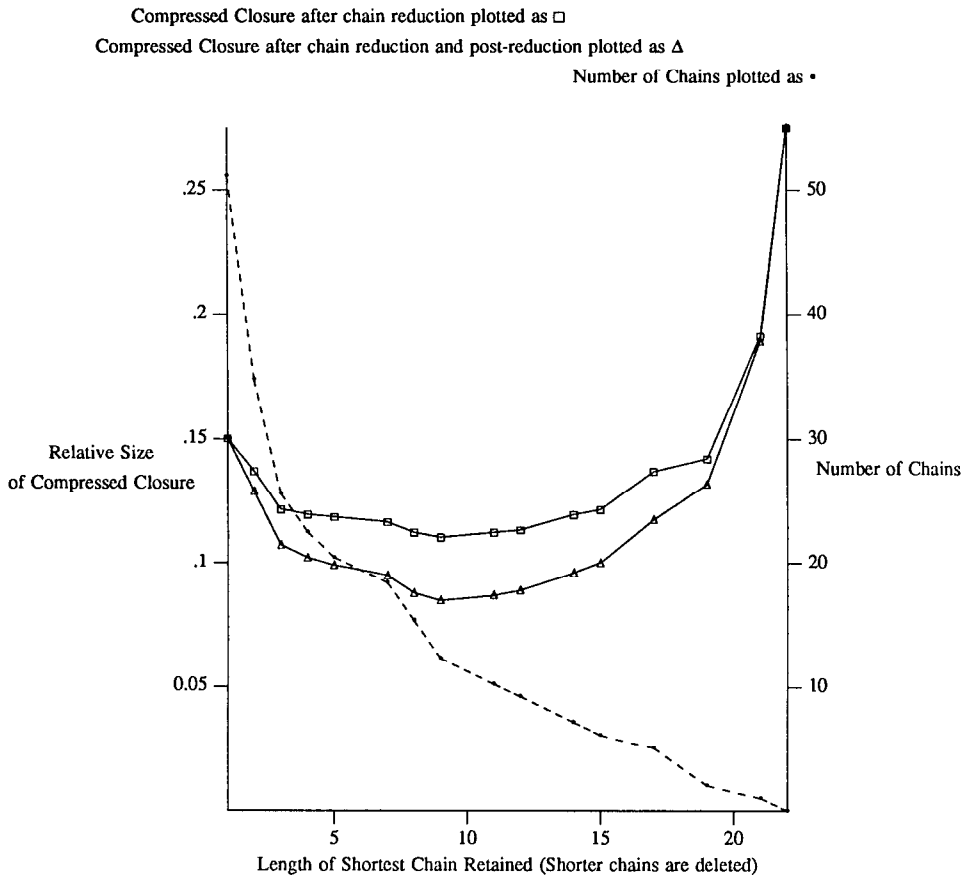


Fig. 9. The effect of varying extent of chain reduction on a 300-node directed acyclic graph of average degree 5 and depth 6, indices having been assigned to the node using Procedure 1 (the Chain Heuristic).

All graphs in Figures 6 and 8 have been drawn to the same scale so that the benefits of chain reduction can be viewed at a glance. The size of the original graph is also plotted for purposes of comparison in Figure 8b. All the remarks made above about the trends in the compression as the graph parameters are varied, as well as the relative merits of the procedures, still apply. It is worth noticing that the relative performance of the procedures has a smaller variance after chain reduction than before. In other words, chain reduction provides a greater benefit when the initial index assignment is poor.

Postreduction compression, as suggested in Section 4, was tried on all the graphs. The size of the compressed closure appeared to be 10–20 percent less than could be obtained with the basic chain reduction alone. However, no new trends were observed with this additional reduction. For the particular graph and index assignment heuristic of Figure 9, the effect of postreduction is plotted. No other graphs with post-reduction are presented in the interests of space.

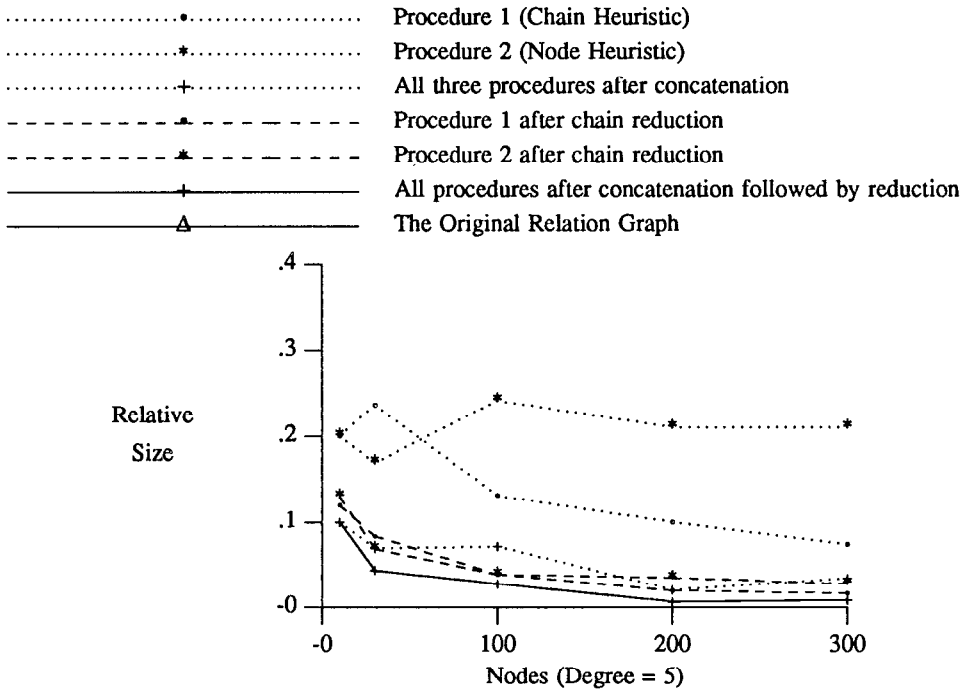


Fig. 10a. Compression of transitive closure as the number of nodes is varied on a random directed graph with cycles.

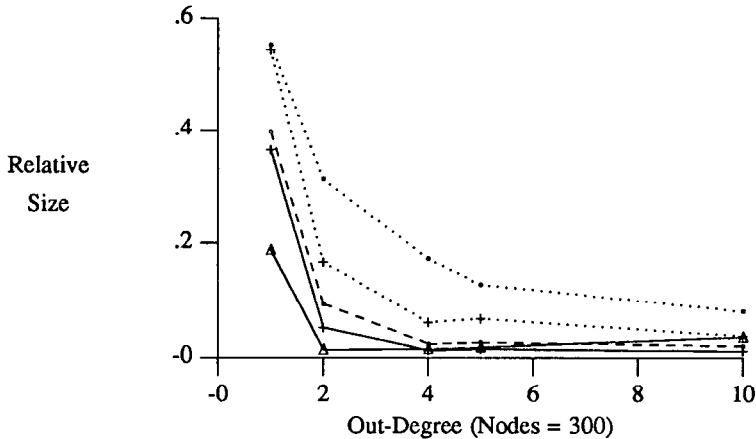


Fig. 10b. Compression of transitive closure as the out-degree is varied on a random directed graph with cycles.

5.3 Graphs with Cycles

Figures 10a and 10b show the relative size of the compressed closure for a graph with cycles as graph parameters are varied. Once more, only a subset of the measurements is plotted. Topological sorting is not possible on a graph with cycles. Procedure 3 once again performs just as well both with and without chain

concatenation. After chain concatenation Procedures 1 and 2 now generate compressions that are roughly as good as those generated by Procedure 3. Therefore, no separate curves are plotted for the procedures after chain concatenation. Curves plotted are thus for three index assignments, with and without chain reduction.

The trends for graphs with cycles are the same as for acyclic graphs as the number of nodes (Figure 10a) or the degree of the graph (Figure 10b) is varied. However, the compression achieved for a random graph is greater than the compression achieved for an acyclic graph with the same parameters. The reason is that chains are “unidirectional” in an acyclic graph, always moving in the direction of nodes that appear later in the topological order. In a general random graph, chains can “wind their way back and forth” within each strongly connected component, thereby covering more nodes.

Comparing the performance of the different heuristics, Procedure 3 did remarkably well. In fact Procedures 1 and 2 after chain concatenation resulted in compressed stores that were approximately equal to what Procedure 3 generated in the first place. Only the curve for Procedure 3 is shown after chain concatenation. Without concatenation, Procedure 1 did somewhat better overall than Procedure 2. The same trends held true after chain reduction as well. The longest chains tended to cover more nodes in graphs with cycles than in acyclic graphs. Minimum storage after chain reduction was achieved with only one or two longest chains retained and all the others are deleted. As in the case of acyclic graphs, post-reduction compression could be used to achieve a further 10–20% savings in the storage required.

It was suggested in Section 3 that one technique for compressing the transitive closure of a random graph with cycles was to collapse its strongly connected components and then to compress the closure of the resulting acyclic graph. For a degree two or greater, even very large graphs usually collapsed into a handful of strongly connected components so that the storage for the closure was dominated by the table associating each node with the strongly connected component it was part of. In other words, the storage required was of the order of the number of nodes in the graph. Whenever the update frequency is low enough for it to be worthwhile to compute strongly connected components for a graph with cycles, this technique is highly recommended.

6. INCREMENTAL CHANGES

Once a compressed transitive closure has been computed and stored, it is possible that changes are made to the underlying database relation, and hence to the transitive closure. As the frequency of such updates increases, maintaining a materialized view (our compressed transitive closure) rapidly becomes impractical if the entire transitive closure and its compression have to be recomputed after every change. However, it may be possible to account for the effect of some updates without having to recompute and recompress the closure. In this section we present some incremental algorithms that can be used to take minor changes into account quickly.

In addition to the effort involved in incorporating the effects of the change in the compressed storage structure, there is another issue as well: the impact of

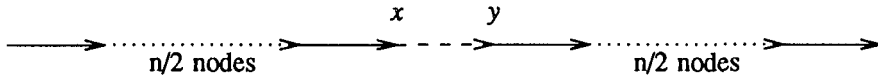


Fig. 11. Addition of an edge to a graph.

such incremental changes on the compression achieved, and hence on the size of the storage structure. We examine this issue in Sections 6.4 and 6.5.

6.1 Addition of an Edge

The addition of a new tuple to the base relation is equivalent to the addition of a directed edge in the underlying graph. Let x be the source, and y the destination node of the new arc xy .⁵

The addition of a single edge can cause $O(n^2)$ edges in the transitive closure of a graph. For example, in Figure 11, the addition of the dashed edge will create paths from each of $n/2$ nodes to the left of the break to each of the $n/2$ nodes to the right, increasing the size of the transitive closure by $n^2/4$ entries. If the full closure is maintained, recording each entry requires at least one unit of effort, so that $\Omega(n^2)$ effort is a lower-bound on the worst-case effort for updating the transitive closure upon incremental addition of an arc.

If chain compression is used, even the update could occur more cheaply. For example, let two chains cover the graph of Figure 11: one for the left half and one for the right half. Upon adding the dashed arc, each node in the left half has to record y , the first node in the right chain, as a successor. In other words, each of $n/2$ nodes records one new entry in their successor lists. In general, if c chains cover the graph, the size of the compressed closure is $O(nc)$, and so is the size of the number of entries generated through the addition of a single arc. Thus there is a $\Omega(nc)$ lower bound on the effort required for an incremental update of the chain compressed transitive closure.

Consider the addition of an arc xy to a graph. Every node that has a path to x , now has a path to y and to every successor of y , by transitivity, due to the addition of xy , and reachability is not affected between any other pair of nodes. Therefore, one should add the (compressed) successor list of y to that of every node that has a path to x . A node that can reach y prior to the addition of xy , can also reach all the successors of y ; therefore there is no need to merge the successor list of y with any such node. On this basis, we now present an algorithm that achieves the $\Omega(nc)$ lower bound on update effort upon adding arc xy .

Incremental Addition Procedure

```

Add-node ( $y, x$ );
Merge ( $y, x$ );
For each node  $n$ 
  If ((there is a path from  $n$  to  $x$ ) && (there is no path from  $n$  to  $y$ ))
  {
    Add ( $y, n$ );
    Merge ( $y, n$ );
  }
```

⁵ Assume for now that x and y are nodes that already exist, with only the edge between them being new. In Section 6.3, we will study the case where x or y is a new node.

where Merge (p, q) is the following procedure to merge the compressed successor list of p with the compressed successor list of q :

```

Merge ( $p, q$ )
{
  For each chain  $k$ 
    Let  $(j, k)$  be the entry in the compressed successor list of  $p$ .
    Add  $((j, k), q)$ ;
}

```

where Add-node (p, q) simply executes Add $((j, k), q)$, for (j, k) being the index pair assigned to node p ; and where Add $((j, k), q)$ adds node (j, k) to the compressed successor list of node q :

```

Add  $((j, k), q)$ 
{
  Let  $(i, k)$  be the entry in the compressed successor list of  $q$  for chain  $k$ .
  (If no entry for chain  $k$ , then assume  $i = \infty$ .)
  If  $(i > j)$  then add  $(j, k)$  to the compressed successor list of  $q$ , and delete  $(i, k)$ .
}

```

Merge (p, q) requires $O(c)$ effort, and is executed once, in the worst case, for each node in the graph. Therefore the total worst case effort is $O(nc)$, matching the lower bound obtained above. In fact, the Procedure can be implemented as a single pass over the entire database, and requires effort that is proportional to the size of the compressed storage structure.

Some simple optimizations are possible. One can perform an initial check to see if y can be reached from x prior to the addition of xy . If so, then no update is required at all.

Also, rather than merge the compressed successor list of y with each node, one could merge a smaller list, called the *modification list*, comprising only the new entries in the compressed successor list of x obtained upon performing a Merge (y, x). That this is correct can be argued as follows:

If node n has a path to x , and the entry for chain k in the compressed successor list of n is (i, k) , and in the compressed successor list of x is (j, k) , then we must have $i \leq j$. Therefore, if y has an entry (l, k) in its compressed successor list such that $j \leq l$, and Merge (y, x) does not modify the entry for chain k in the successor list of x , then we must also have $i \leq l$, so that a Merge (y, n) will not modify the entry for chain k in the compressed successor list for n . Hence it is enough to merge the modification list, rather than the entire compressed successor list of y , with all such nodes n . Thus the optimization is correct.

To avoid having to examine every node, one could store at each node not only the compressed successor list but also the compressed predecessor list. Then the merging of the modification list need be done only by all the nodes indicated in the predecessor list of x . However, the predecessor lists themselves must also be updated, requiring a symmetric update in all the successors of y .

Even though the addition of a single arc can cause significant effort to be expended, it is possible to devise data structures with the help of which a sequence of arc additions is not very expensive. A cost measure in such cases is called the *amortized cost*. A technique has been proposed in [21] to achieve an $O(n)$ amortized update cost for incremental addition of a single edge to a graph. This

technique can be adapted in a straightforward fashion to be applicable even in the presence of chain compression.

6.2 Deletion of an Arc

If an arc xy is deleted, check whether any of the remaining immediate successors of the source node, x , of this arc can reach the destination, y , of the deleted arc. If such a successor is found, then x can still reach y (though not directly any more) and the deletion of the arc does not alter the transitive closure. If not, we have two cases.

If x and y belong to the same chain, the deletion of xy causes one chain to be split in two and reindexing is required. Suppose that the original second index was j . Then the nodes to the left of the break still belong to chain j whereas the nodes to the right belong to a new chain k . Let (i, j) be the index for node y , the destination of the arc deleted. Then all nodes that have (m, j) in their compressed successor lists for some $m \geq i$, have to change their lists to refer to (m, k) rather than (m, j) . (Obviously, these nodes cannot reach any node in chain j , so nothing new need be added). For nodes with (m, j) in their successor lists for some $m < i$, add (i, k) (the new name for the old (i, j)) to the successor list (retaining the (m, j)). We thus create a situation in which the deleted arc is between nodes on two different chains and then apply the procedure below.

If x and y belong to different chains (originally, or after the reindexing), recompute the compressed successor list of x by merging (and deleting dominated nodes from) the compressed successor lists of all the immediate successors of x (other than y). Proceeding in reverse topologically sorted order, similarly recompute the compressed successor lists of all nodes that precede x in a topological sort.

If a topological sort is not available on an acyclic graph, or is impossible (because the graph has cycles), a single linear pass over the nodes does not suffice. In that case a depth-first search algorithm is required. The successor list of a node can be recomputed (and thereby rendered *current*) once the successor lists of all its immediate successors are current. The successor list of a node, w , is current if it has been recomputed, or if w cannot reach x . To ensure that the recursion terminates even for a graph with cycles, the depth-first search must be enhanced with a cycle detection mechanism. For this purpose, the strongly-connected component algorithm proposed in [32], can be used to maintain an explicit recursion stack, and each time a cycle is detected to pop all nodes in the cycle from it, assigning to each node the union of the successor lists of all.

The effort involved in the above algorithm is $O(Ec)$, where E is the number of edges in the graph, and c is the number of chains used to cover it. The reason is that, in the worst case, for each node one has to merge the compressed successor lists of its immediate successors. Each compressed successor list is $O(c)$ long, and the total number of such lists merged equals the number of immediate successors of all nodes in the graph, which is the number of edges in the graph. This effort is only slightly better in worst case performance than the complete recomputation of the transitive closure. However, in many instances, the effort could be significantly less.

Incremental deletions are a problem not just for our compression technique, but for transitive closures in general. The reason is that when a new arc is added, only the closure information is required to determine the new paths that can be devised as a consequence of the arc addition. However, when an arc is deleted, the closure information by itself is not enough: one needs in addition to know the actual paths that were followed to obtain each entry in the closure, so that one can determine which of these paths are no longer valid, if any. As such, the incremental techniques presented here are essentially a recomputation of the closure for every node that can reach x , the source node of the deleted arc. Since incremental updates can be expensive upon edge deletion, the compressed transitive closure technique suggested in this paper is likely to be appropriate only when such deletion updates are infrequent.

If updates due to edge deletions can be deferred and batched, the effort to update need be expended just once on a set of edge deletions. The only difference now is that a node is marked *current* iff it has been recomputed or cannot reach the source nodes of *any* of the deleted arcs.

6.3 Addition and Deletion of a Node

In a relational database stored as a set of tuples representing arcs it is not possible to add or delete a node in isolation. The addition or deletion of a node can only be the side effect of the addition or deletion of an arc. However, one may conceptually like to think of the addition or deletion of a node as an entity.

If a new node, x , has associated with it an arc xy , then only its successor list will get created, and no other successor list will change. However, an index pair must be assigned to it. If the first node in some chain happens to be a successor of x , then add x to the beginning of the chain. Otherwise, the node is assigned a new chain. If a new node, y , has associated an arc xy , then it currently has an empty successor list. However, it must itself be added to the successor lists of all its predecessors. If any of its predecessors is the last node on some chain, the new node can be assigned that chain. Otherwise a new chain is created. If chain reduction is in use, rather than create a new chain, treat the new node as an off-chain node. Note that the index assignment to a new node here is exactly the same as would have been obtained with the node heuristic, if the new node were the last one to be considered.

Deletion of a node can be interpreted in two ways. One is to retain all the reachability between nodes that go through this node, and the other is to completely remove the node and all arcs incident on it.

The first interpretation leads to the addition of all the immediate predecessors of the deleted node to the list of immediate predecessors of its immediate successors, and vice versa. Also, references to the deletion node (i, j) in the compressed successor list of any node should be replaced by (k, j) where $k > i$ and there is no node (m, j) for any $i < m < k$.

The second interpretation requires the deletion of each arc incident upon the node, one after another, following the procedure described in Section 6.2, with the update of the closure batched over all the edge deletions. Thereafter, the node neither reaches nor can be reached by any other and can safely be deleted with no change to the successor list of any node.

6.4 Incremental Compression

The reader will have noticed that all of the incremental changes listed above can only add to the number of chains in the graph, and hence to the storage requirements, and can never reduce them. The incremental algorithms have been designed with efficiency of update as the primary objective, and not efficiency of storage. After several incremental changes have accumulated it may be worthwhile to reassign all indices from scratch and compress the closure. (It is still not necessary to recompute the transitive closure).

A reduction in the number of chains could occur whenever an arc is added that creates a path between two nodes that were previously mutually unreachable. Such a reduction would in general require finding a new chain cover, and a complete reindexing of the entire graph. The old indices, and the compressed successor lists according to the old indices (including all updates that may have occurred thus far), are retained while the new indices are being assigned. Thereafter, each compressed successor list can be expanded into its full form and recompressed according to the new indices, one node at a time.

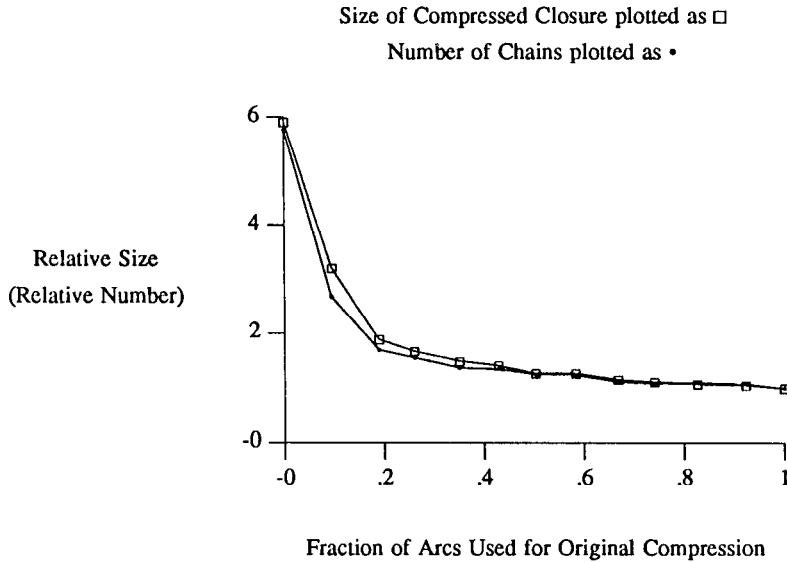
Even less work is possible in some cases. If an arc is added from a node that is the last in its chain (or a node that is reachable from a node that is last in its chain), to a node that is first in its chain (or a node that can reach a node that is first in its chain), then the two chains in question can be merged into a single chain. This procedure is the same as the chain concatenation discussed in Section 3, except that it is carried out after all the nodes already have compressed successor lists. Appropriate reindexing of the nodes on this chain is required. For example, with the addition of the dashed edge in Figure 11, the two chains can be merged into one. The storage required after arc addition is now no more than the storage required before. The effort for the update remains $O(n)$, the same as in Section 6.1, due to the reindexing and compressed successor list update of $n/2$ nodes to the right of the break.

6.5 Experimental Evaluation

In Section 6, we have so far presented update algorithms with reasonable time complexities in the face of incremental additions to the graph. As pointed out in Section 6.4, these algorithms have been devised with efficiency of update in mind. In this subsection, we explore the cost in terms of increased storage due to such incremental additions.

For a number of different graphs, we took a fraction of the edges in the graph and performed an index assignment and compression based upon these edges only. Thereafter, the remaining edges were added incrementally, one by one, and the compressed transitive closure updated each time. No index assignment, reindexing, or incremental compression was performed; for each arc addition, the incremental procedure described in Section 6.1 was executed. The size of the compressed closure was measured once all edges had been input, and this size compared against the size that would have been obtained if the entire graph were compressed at once.

Figure 12 shows the results obtained for one particular graph. Similar trends were observed for several other graphs for which the experiment was run. On the X-axis is the fraction of the arcs in the graph that were used for the initial



(Nodes = 300 Degree = 5, Depth = 3, Node Heuristic without Concatenation or Reduction)

Fig. 12. Incremental update of the compressed transitive closure of a directed acyclic graph.

compression. At the left-end of the scale, no edges were available initially, so each node was formed into a new chain, and no compression is achieved. At the right end of the scale, all edges have been included in the initial compression and no incremental updates have been performed. Two different curves have been plotted. One shows the size of the final compressed closure relative to the size of the compressed closure that can be obtained if the entire graph were compressed at once. The other curve shows the number of chains used to cover the graph relative to the number of chains required if the entire graph were compressed at once. Both curves show parallel trends.

Consider the value 0.5 on the X-axis. This represents a point at which a graph was compressed and then its size doubled by means of arcs that were incrementally added. The size of the compressed closure storage structure obtained after all the incremental updates is roughly 1.2 times the size of the compressed storage structure that could be obtained if the index assignment and compression were performed once after all the edges were known. 20% seems like a small penalty to pay for considerable update activity. In fact, the graph indicates that 3-4 times as many edges can be added as were originally present without seriously degrading the compression obtained. As such, the suggested strategy, permitting incremental additions and then recompressing periodically after activity accumulates, will probably provide good storage compression unless the update frequency is extremely high.

7. COMPUTING TRANSITIVE CLOSURE

The computation of transitive closure is an expensive proposition. Thus far in this paper we have assumed that the transitive closure has already been computed

before the successor lists are compressed. In this section we show how it may be worthwhile to compress the successor lists continuously as the computation of the transitive closure progresses, and thereby to reduce the effort required to compute the transitive closure itself.

There are three distinct operations that must be carried out to obtain a compressed transitive closure:

- (i) Assign indices to each node. This assignment is independent of the other operations provided one does not use a heuristic that requires knowledge of the full transitive closure.
- (ii) Compute the transitive closure. This computation is also independent of the other operations. Indeed transitive closures have hitherto always been computed with no reference to compression at all.
- (iii) Compress successor lists for each node. Indices must have been assigned before any compression can take place, since the compression relies upon index comparison. This operation cannot precede the computation of the transitive closure, which creates the large successor lists to be compressed.

Transitive closure computation techniques typically add to the successor list of each node incrementally as they “discover” more and more nodes that can be reached. For each node in the given graph, the (immediate) successor list can first be compressed, and then the compression maintained as each new successor is discovered in the course of closure computation. To maintain compression in the successor list of some node p , every time a new successor, with indices (i, j) , is discovered, examine the entry for chain j in the successor list of p . Let this entry be (k, j) . If $k \leq j$, no update is required. Otherwise,⁶ substitute (i, j) for (k, j) .

We simulated the computation of transitive closure using continuous compression of successor lists as discussed above. We used only directed acyclic graphs and assumed that the nodes were already in a topologically sorted order. The current best technique for computing transitive closure for most graphs is the Blocked Row algorithm [2]. In general two passes are required to compute the closure of a relation using the Blocked Row algorithm. However, if the nodes are topologically sorted, then only one pass suffices. We used the one-pass Blocked Row algorithm to compute the closure. The algorithm examines each node in the graph exactly once following a reverse topologically sorted order. When a node is examined, its successor list is updated by adding to it the successor lists of all its immediate successors, removing duplicates if any. Index assignment was done according to Procedure 2, which permitted indices to be assigned to each node as it was examined.

Figure 13 shows how the I/O requirements were reduced due to the use of continuous compression for a 1000 node acyclic graph. Each tuple was assumed to require 10 Bytes of storage. The memory size was assumed to be 100,000 Bytes. Results for other memory sizes, and other types and sizes of graphs were qualitatively similar. We assumed that the relation was large and resident on disk so that I/O dominated the effort to compute the closure. However, the same

⁶ Assume $k = \infty$, for i has no known successors in chain j .

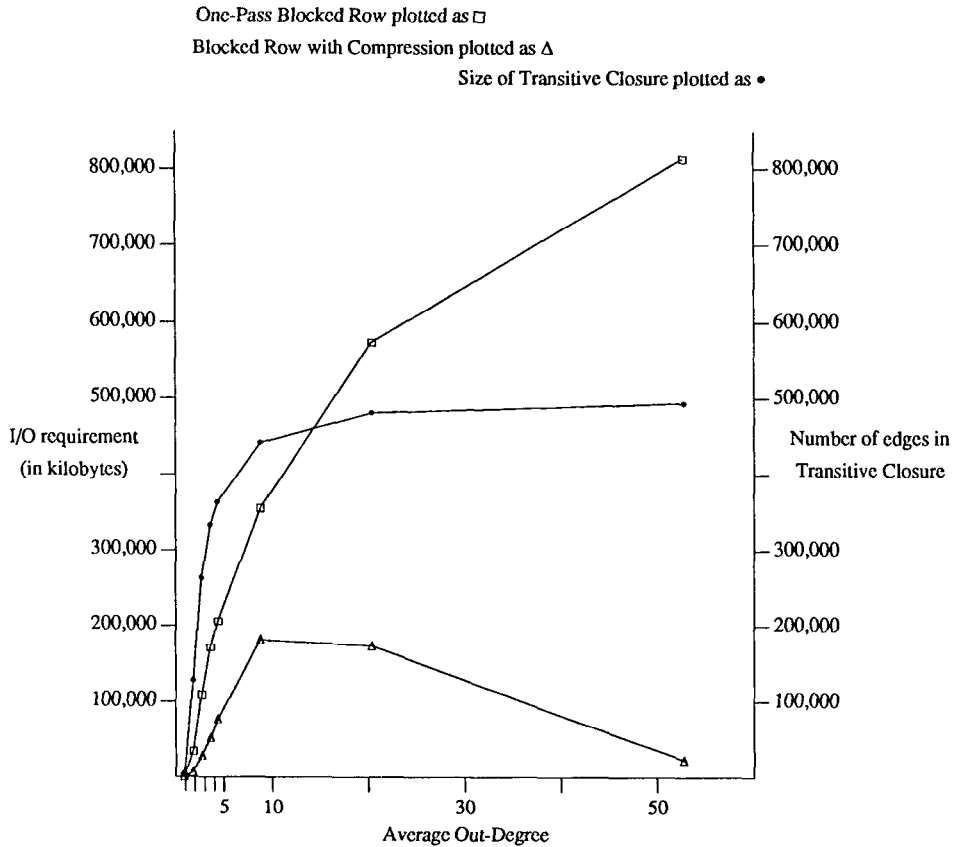


Fig. 13. Savings in I/O due to continuous compression during computation of transitive closure on a 1000 node acyclic graph with depth 10, with a tuple size of 10 bytes, and a memory size of 100,000 bytes.

qualitative results apply even for a completely memory resident algorithm on a much smaller database, on account of the reduced effort now required for eliminating duplicates from shorter successor lists. In fact, we expect that similar results would also be obtained for other algorithms to compute transitive closure, such as the "seminative" algorithm.

The number of edges in the transitive closure has also been plotted in Figure 13 for purposes of comparison. As in Figure 7, this grows rapidly at first, and then asymptotically approaches the 500,000 edge limit. The effort required to compute the transitive closure grows even more rapidly, and continues to grow even after the size of the transitive closure itself has leveled off, due to the fact that one is now starting with a larger initial relation and discovering most of the successors earlier, requiring a larger relation to be manipulated during most of the closure computation. The effort to compute closure with continuous compression shows the same general shape of curve as the size of the compressed closure (shown in Figure 7), rising at first and then dropping off, as fewer and fewer

chains are required to cover the graph. The improvement over the case without continuous compression is greatest for graphs of large degree.

8. CONCLUSIONS

If it is expected that multiple fixed-point queries will be made on a database, it may be worthwhile to precompute the transitive closure, using techniques such as those presented in [2], and store the result. Indeed, such an approach has been followed, for example in [36]. In this paper we have presented a technique for efficiently storing the transitive closure of a database relation. Using the compression techniques suggested in this paper, a precompute and materialize strategy becomes more attractive.

It is possible to determine not only whether a specified point x can be reached from a specified point y , but also all nodes that can be reached from a specified node y . Finding all nodes that can reach a specified node x is not conveniently possible in the successor-oriented storage structure developed, and a separate but symmetric predecessor-oriented structure is required to handle such a query as a simple look-up. However, if the entire transitive closure database were stored rather than just the compressed structure, every tuple in the database would have to be examined to find all the nodes that can reach a specified node x unless an index-structure has already been built on this field. Our compressed storage structure can be thought of in the same light as an index structure, such as a B-tree, on a large relation. It allows one to answer efficiently only those queries for which the index structure was built; others require a complete scan of the (compressed) database. Whenever updates occur to the database, the index structure also has to be updated, and good incremental algorithms are required. Since the compressed database is much smaller than the full transitive closure database, even inefficient procedures such as a complete scan will work faster with compression than without.

One drawback of the compression technique presented in this paper is that path information is not maintained. Thus, if the problem is to determine shortest path rather than pure reachability, the results presented in this paper are not directly applicable. However, chain compression can be used to augment path encoding structures [5].

A large part of the cost of computing a transitive closure is in the transfer of data between the disk and main memory. If the closure is compressed "on the fly" as it is being computed, a more efficient (compressed) transitive closure computation technique could emerge. For the specific case of an acyclic graph with the nodes already in topologically sorted order, we have indicated how this might be done. Further work is in progress to devise similar combined techniques in general.

ACKNOWLEDGMENTS

I would like to thank Yehuda Afek for providing the reference to Dilworth's Theorem and the relevant graph theory literature, Rakesh Agrawal, Shaul Dar, and anonymous referees for their useful comments on earlier versions of this paper, and Ron Brachman for his encouragement. Some of the software used for

the experiments was adapted from software previously developed, for a different purpose, by me, jointly with Rakesh Agrawal.

REFERENCES

1. AGRAWAL, R. Alpha: An extension of relational algebra to express a class of recursive queries. In *Proceedings IEEE 3rd International Conference on Data Engineering* (Los Angeles, Feb. 1987), 580–590.
2. AGRAWAL, R., AND JAGADISH, H. V. Direct transitive closure algorithms: Design and performance evaluation. *ACM Trans. Database Syst.* 15, 3 (Sept. 1990).
3. AGRAWAL, R., AND JAGADISH, H. V. Multiprocessor transitive closure algorithms. In *Proceedings of International Symposium on Databases in Parallel and Distributed Systems* (Austin, Tex., Dec. 1988), 56–66.
4. AGRAWAL, R., BORGIDA, A., AND JAGADISH, H. V. Efficient management of transitive relationships in large data and knowledge bases. In *Proceedings ACM-SIGMOD 1989 International Conference on Management of Data* (Portland, Ore., May–June 1989), 253–262.
5. AGRAWAL, R., AND JAGADISH, H. V. Materialization and incremental update of path information. In *Proceedings IEEE 5th International Conference on Data Engineering* (Los Angeles, Feb. 1989), 374–383.
6. BANCILHON, F. Naive evaluation of recursively defined relations. Tech. Rep. DB-004-85, MCC, Austin, Tex., 1985.
7. BANCILHON, F., AND RAMAKRISHNAN, R. An amateur's introduction to recursive query processing strategies. In *Proceedings ACM-SIGMOD 1986 International Conference on Management of Data* (Washington, D.C., May 1986), 16–52.
8. BANERJEE, J., KIM, W., KIM, S.-J., AND GARZA, J. F. Clustering a DAG for CAD databases. *IEEE Trans. Softw. Eng.* SE-14 11 (Nov. 1988), 1684–1699.
9. BEERI, C., KANELLAKIS, P., BANCILHON, F., AND RAMAKRISHNAN, R. Bounds on the propagation of selections into logic programs. In *Proceedings 6th Symposium Principles of Database Systems* (San Diego, Calif., March 1987), 214–226.
10. BITTON, D., DEWITT, D. J., AND TURBYFILL, C. Benchmarking database systems—A systematic approach evaluation. In *Proceedings 9th International Conference on Very Large Data Bases*. (Florence, Oct. 1983).
11. BLAKELEY, J. A., LARSON, P.-A., AND TOMPA, F. W. Efficiently updating materialized views. In *Proceedings ACM-SIGMOD 1986 International Conference on Management of Data* (Washington, D.C., May 1986), 61–71.
12. BRACHMAN, R. J., AND SCHMOLZE, J. G. An overview of the KL-ONE knowledge representation system. *Cognitive Sci.* 9 2 (April–June 1985), 171–216.
13. DEVANBU, P., AND AGRAWAL, R. Moving selections into a class of least fixpoint queries. In *Proceedings IEEE 4th International Conference on Data Engineering* (Feb. 1987).
14. DILWORTH, R. P. A decomposition theorem for partially ordered sets. *Ann. Math.* 51, (1950), 161–166.
15. DINIC, E. A. Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Math. Dokl.* 11 (1970), 1277–1280.
16. GUTTMAN, A. New features for relational database systems to support CAD applications. Ph.D. dissertation, Computer Sciences Dept., Univ. of California, Berkeley, June 1984.
17. HANSON, E. A performance analysis of view materialization strategies. In *Proceedings ACM-SIGMOD 1987 International Conference on Management of Data* (San Francisco, May 1987), 440–453.
18. HULL, R., AND KING, R. Semantic database modeling: Survey, applications, research issues. *ACM Comput. Surv.* 19 3 (Sept. 1987), 210–260.
19. IOANNIDIS, Y. E. On the computation of the transitive closure of relational operators. In *Proceedings 12th International Conference on Very Large Data Bases* (Kyoto, Aug. 1986), 403–411.
20. IOANNIDIS, Y. E., AND RAMAKRISHNAN, R. An efficient transitive closure algorithm. In *Proceedings 14th International Conference on Very Large Data Bases* (Long Beach Calif., Aug.–Sept. 1988), 382–394.

21. ITALIANO, G. F. Amortized efficiency of a path retrieval data structure. *Theor. Comput. Sci.* 48 (1986), 273–281.
22. JAGADISH, H. V., AGRAWAL, R., AND NESS, L. A study of transitive closure as a recursion mechanism. In *Proceedings ACM-SIGMOD 1987 International Conference on Management of Data* (San Francisco, May 1987), 331–344.
23. JAGADISH, H. V. Incorporating hierarchy in a relational model of data. In *Proceedings ACM-SIGMOD 1989 International Conference on Management of Data* (Portland, Ore., May–June 1989), 78–87.
24. FORD, L. R., JR., AND FULKERSON, D. R. *Flows in Networks*. Princeton University Press, Princeton, N.J., 1962.
25. KARZANOV, A. V. Determining the maximal flow in a network by the method of preflows. *Soviet Math. Dokl.* 15 (1974), 434–437.
26. KIFER, M., AND LOZINSKII, E. L. Filtering data flow in deductive databases. In *Proceedings of International Conference on Database Theory* (Rome, Sept. 1986).
27. KOENIG, S., AND PAIGE, R. A transformational framework for the automatic control of derived data. In *Proceedings 7th International Conference on Very Large Data Bases* (1981), 306–318.
28. LU, H. New strategies for computing the transitive closure of a database relation. In *Proceedings 13th International Conference on Very Large Data Bases* (Brighton, England, Sept. 1987).
29. MALHOTRA, V. M., PRAMODH KUMAR, M., AND MAHESHWARI, S. N. An $O(|V|^3)$ Algorithm for finding maximum flow in networks. *Inf. Process. Lett.* 7, 6 (Oct. 1978), 277–278.
30. MEHLHORN, K. *Graph Algorithms and NP-Completeness*, Monographs on Theoretical Computer Science, Springer Verlag, 1984.
31. ROSENTHAL, A., HEILER, S., DAYAL, U., AND MANOLA, F. Transversal recursion: A practical approach to supporting recursive applications. In *Proceedings ACM-SIGMOD 1986 International Conference on Management of Data* (Washington, D.C., May 1986), 166–176.
32. TARJAN, R. Depth-first search and linear graph algorithms. *SIAM J. Comput.* 1, 2 (June 1972), 146–160.
33. TOUREZKY, D. S. *The Mathematics of Inheritance Systems*. Morgan Kaufmann, Los Altos, Calif., 1987.
34. VALDURIEZ, P., AND BORAL, H. Evaluation of recursive queries using join indices. In *Proceedings 1st International Conference on Expert Database Systems* (Charleston, S.C., April 1986), 197–208.
35. VALDURIEZ, P., AND KHOSHAFIAN, S. Parallel evaluation of the transitive closure of a database relation. *Intl. J. Parallel Programm.* 17, 1 (Feb. 1988), 19–42.
36. WALTER, I. M., LOCKEMANN, P. C., AND NAGEL, H.-H. Database support for knowledge-based image evaluation. In *Proceedings of the 13th International Conference on Very Large Databases* (Brighton, England, 1987), 3–11.
37. ZLOOF, M. M. Query-by-example: Operations on the transitive closure. Tech. Rep. RC 5526, IBM, Yorktown Heights, N. Y., 1975.

Received September 1988; revised April 1989; accepted August 1989